

교과목 : 분산처리

1. CUDA 12.6 설치 및 사용 환경 설정

2025학년도 1학기
옥수열

Computer & Ai

Department of Computer Engineering

1. Microsoft Windows용 CUDA 설치 가이드

■ Microsoft Windows 시스템에서 CUDA 툴킷을 설치하기 위한 지침 소개

특징

- CUDA는 NVIDIA에서 개발한 병렬 컴퓨팅 플랫폼 및 프로그래밍 모델.
- 그래픽 처리 장치(GPU)의 성능을 활용하여 컴퓨팅 성능을 극적으로 향상시킬 있음.
- CUDA는 몇 가지 설계 목표를 염두에 두고 개발되었음
 - 병렬 알고리즘을 간단하게 구현할 수 있도록 C와 같은 표준 프로그래밍 언어에 대한 작은 확장 세트를 제공.
 - CUDA C/C++를 사용하면 프로그래머는 알고리즘 구현에 시간을 소비하는 대신 알고리즘의 병렬화 작업에 집중할 수 있음.
 - 애플리케이션이 CPU와 GPU를 모두 사용하는 이기종 연산을 지원.
 - 애플리케이션의 직렬 부분은 CPU에서 실행되고 병렬 부분은 GPU로 오프로드됨. 따라서 CUDA는 기존 애플리케이션에 점진적으로 적용 가능.
 - CPU와 GPU는 자체 메모리 공간을 가진 별도의 장치로 취급됨. 이 구성은 또한 메모리 리소스에 대한 경합 없이 CPU와 GPU에서 동시에 계산할 수 있음.

1. Microsoft Windows용 CUDA 설치 가이드

■ Microsoft Windows 시스템에서 CUDA 툴킷을 설치하기 위한 지침 소개

특징

- ➔ CUDA 지원 GPU에는 수천 개의 컴퓨팅 스레드를 총체적으로 실행할 수 있는 수백 개에서 수천개의 코어가 있음. 이러한 코어에는 레지스터 파일과 공유 메모리를 비롯한 공유 리소스가 존재.
- ➔ 온칩 공유 메모리를 사용하면 이러한 코어에서 실행 중인 병렬 작업에서 시스템 메모리 버스를 통해 데이터를 전송하지 않고도 데이터를 공유할 수 있음.

1. Microsoft Windows용 CUDA 설치 가이드

시스템 요구사항

- ❖ 시스템에서 CUDA를 사용하려면 다음이 설치되어 있어야 함
 - ➔ CUDA 지원 GPU, gcc 컴파일러 및 툴체인이 지원되는 Linux 버전
 - ➔ NVIDIA CUDA 툴킷(<https://developer.nvidia.com/cuda-downloads> 에서 제공)
 - ➔ 지원되는 Microsoft Windows 운영 체제®
 - ✓ Microsoft Windows 11 21H2(SV1), Microsoft Windows 11 22H2(SV2), Microsoft Windows 11 23H2, Microsoft Windows 10 22H2, Microsoft Windows Server 2022

Table 1: Windows Compiler Support in CUDA 12.6 Update 3

Compiler*	IDE	Native x86_64	Cross-compilation (32-bit on 64-bit)	C++ Dialect
MSVC Version 193x	Visual Studio 2022 17.x	YES	Not supported	C++14 (default), C++17, C++20
MSVC Version 192x	Visual Studio 2019 16.x	YES		C++14 (default), C++17
MSVC Version 191x	Visual Studio 2017 15.x (RTW and all updates)	YES		C++14 (default), C++17

- ➔ Visual Studio 2015에 대한 지원은 릴리스 11.1에서 중단되었으며, Visual Studio 2017에 대한 지원은 릴리스 12.5에서 중단됨.
- ➔ 32비트 컴파일 네이티브 및 교차 컴파일은 CUDA 12.0 이상 툴킷에서 제거되었음.
- ➔ 32비트 컴파일을 위해서는 이전 릴리스의 CUDA 툴킷을 사용.
- ➔ CUDA 드라이버는 Ada까지 GeForce GPU에서 32비트 애플리케이션 바이너리 실행을 계속 지원함.
- ➔ Ada는 32비트 애플리케이션에 대한 드라이버 지원을 제공하는 마지막 아키텍처가 됨.
- ➔ Hopper는 32비트 애플리케이션을 지원하지 않음.
- ➔ X86_64 Windows에서 x86 32비트 애플리케이션 실행에 대한 지원은 제한적으로 제공됨: [CUDA 드라이버](#), [CUDA 런타임\(cudart\)](#), [CUDA 수학 라이브러리\(math.h\)](#)

1. Microsoft Windows용 CUDA 설치 가이드

CUDA 개발 설치 Tools

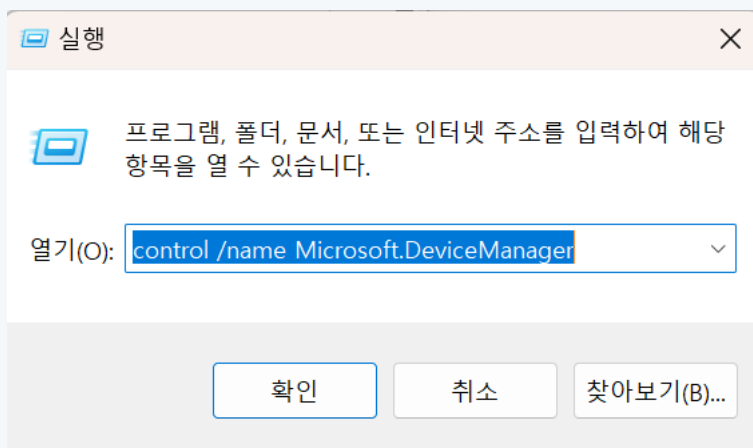
❖ 적절한 버전의 Windows를 실행하는 시스템에서 CUDA 개발 툴을 설치하는 방법은 몇 가지 간단한 단계로 구성:

- 시스템에 CUDA 지원 GPU가 있는지 확인.
- NVIDIA CUDA 툴킷을 다운로드.
- NVIDIA CUDA 툴킷을 설치.
- 설치된 소프트웨어가 올바르게 실행되고 하드웨어와 통신하는지 테스트.

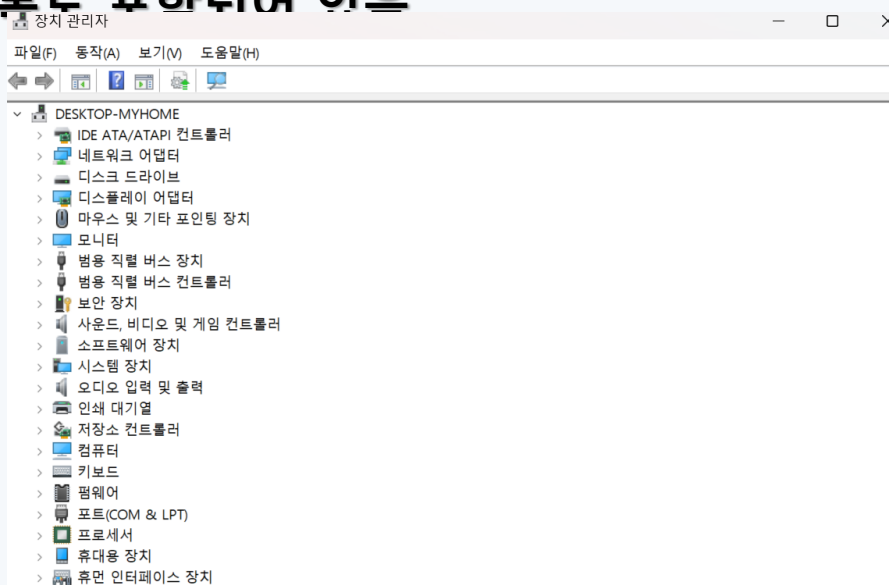
1. Microsoft Windows용 CUDA 설치 가이드

- 시스템에 CUDA 지원 GPU가 있는지 확인.

- ❖ Windows 장치 관리자의 디스플레이 어댑터 섹션에서 CUDA 지원 GPU가 있는지 확인 가능.
- ❖ 여기에서 그래픽 카드의 공급업체 이름과 모델을 확인할 수 있음.
- ❖ <https://developer.nvidia.com/cuda-gpus> 에 나열된 NVIDIA 카드를 가지고 있다면 해당 GPU는 CUDA를 지원함.
- ❖ CUDA 툴킷의 릴리스 노트에는 지원되는 제품 목록도 포함되어 있음



control /name Microsoft.DeviceManager



1. Microsoft Windows용 CUDA 설치 가이드

■ NVIDIA CUDA 툴킷을 다운로드

❖ NVIDIA CUDA 툴킷은 <https://developer.nvidia.com/cuda-downloads>에서 확인 가능.

❖ 사용 중인 플랫폼과 다음 인스톨러 형식 중 하나를 선택:

1. 네트워크 설치 프로그램(network): 설치에 필요한 패키지를 나중에 다운로드하는 최소한의 설치 프로그램. 인스톨러 선택 단계에서 선택한 패키지만 다운로드됨. 이 인스톨러는 다운로드 시간을 최소화하려는 사용자에게 유용.
2. 전체 설치 관리자(local): CUDA 툴킷의 모든 구성 요소가 포함되어 있으며 추가 다운로드가 필요 없는 인스톨러. 이 인스톨러는 네트워크 액세스가 부족한 시스템이나 기업 배포에 유용.

Operating System

Linux

Windows

Architecture

x86_64

Version

10

11

Server 2022

Installer Type

exe (local)

exe (network)

- CUDA 툴킷은 CUDA 애플리케이션을 생성, 빌드 및 실행하는데 필요한 CUDA 드라이버와 툴은 물론 라이브러리, 헤더 파일 및 기타 리소스를 설치함.
- 다운로드 확인
다운로드는 <https://developer.download.nvidia.com/compute/cuda/12.6.3/docs/sidebar/md5sum.txt>에 게시된 MD5 체크섬과 다운로드한 파일의 체크섬을 비교하여 확인할 수 있습니다. 두 체크섬 중 하나가 다르면 다운로드한 파일이 손상된 것이므로 다시 다운로드해야 함.

1. Microsoft Windows용 CUDA 설치 가이드

■ NVIDIA CUDA 툴킷을 설치.

📌 Note

- ❖ CUDA가 작동하려면 드라이버와 툴킷이 설치되어 있어야 함. 독립 실행형 드라이버를 설치하지 않은 경우 NVIDIA CUDA 툴킷에서 드라이버를 설치하세요.
- ❖ 설치가 시작된 후 Windows 업데이트가 시작되면 설치가 실패할 수 있음. Windows 업데이트가 완료될 때까지 기다렸다가 설치를 다시 시도하세요.

● 그래픽 설치

✓ CUDA 설치 프로그램을 실행하고 화면의 지시에 따라 CUDA 소프트웨어를 설치.

● 자동 설치

- ✓ -s 플래그를 사용하여 패키지를 실행하면 인스톨러를 자동 모드로 실행할 수 있음
- ✓ 모든 패키지 대신 특정 하위 패키지를 설치하도록 추가 파라미터를 전달할 수 있음. 모든 하위 패키지 이름 목록은 아래 표를 참조하세요.

Table 2: Possible Subpackage Names	
Subpackage Name	Subpackage Description
Toolkit Subpackages (defaults to C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.6)	
cuda_profiler_api_12.6	CUDA Profiler API.
cudart_12.6	CUDA Runtime libraries.
cubdump_12.6	Extracts information from cubin files.
capt_12.6	The CUDA Profiling Tools Interface for creating profiling and tracing tools that target CUDA applications.
cuxfilt_12.6	The CUDA cu++filt demangler tool.
demo_suite_12.6	Prebuilt demo applications using CUDA.
documentation_12.6	CUDA HTML and PDF documentation files including the CUDA C++ Programming Guide, CUDA C++ Best Practices Guide, CUDA library documentation, etc.
nvcc_12.6	CUDA compiler.
nvdisasm_12.6	Extracts information from standalone cubin files.
nvfatbin_12.6	Library for creating fatbinaries at runtime.
nvjitlink_12.6	nvjitlink library.
nmml_dev_12.6	NVML development libraries and headers.
nvprof_12.6	Tool for collecting and viewing CUDA application profiling data from the command-line.
nvprune_12.6	Prunes host object files and libraries to only contain device code for the specified targets.
nrtc_12.6	NVRTC runtime libraries.
nvtr_dev_12.6	
sanitizer_12.6	Compute Sanitizer API.
thrust_12.6	CUDA Thrust.
cublas_12.6	cuBLAS runtime libraries.
cublas_dev_12.6	
cufft_12.6	cuFFT runtime libraries.
cufft_dev_12.6	
curand_12.6	cuRAND runtime libraries.
curand_dev_12.6	
cusolver_12.6	cuSOLVER runtime libraries.
cusolver_dev_12.6	
cusparse_12.6	cuSPARSE runtime libraries.
cusparse_dev_12.6	
npp_12.6	NPP runtime libraries.
npp_dev_12.6	
nvjpeg_12.6	nvJPEG libraries.
nvjpeg_dev_12.6	
night_compute_12.6	Night Compute.
night_systems_12.6	Night Systems.
night_vse_12.6	Installs the Night Visual Studio Edition plugin in all VS.
occupancy_calculator_12.6	Installs the CUDA_Occupancy_Calculator tool.
visual_studio_integration_12.6	Installs CUDA project wizard and builds customization files in VS.
Driver Subpackages	
DisplayDriver	The NVIDIA Display Driver. Required to run CUDA applications.

1. Microsoft Windows용 CUDA 설치 가이드

- NVIDIA CUDA 툴킷을 설치.

❖ 예를 들어 컴파일러와 드라이버 구성 요소만 설치하려면 다음과 같이 하세요:

- <패키지명>.exe -s nvcc_12.1 Display.Driver

❖ 재부팅이 필요한 경우에도 설치 또는 제거 후 자동으로 재부팅하지 않으려면 -n 옵션을 사용함.

❖ 수동으로 파일 추출 및 검사하기

- 엔터프라이즈 배포와 같이 설치 가능한 파일을 직접 추출하거나 검사하거나 설치 전에 파일을 찾아보는 것이 바람직한 경우도 있음.
- 전체 설치 패키지는 7-zip 또는 WinZip과 같이 LZMA 압축 방식을 지원하는 압축 해제 도구를 사용하여 추출할 수 있음.
- 압축이 풀리면 CUDA 툴킷 파일은 CUDAToolkit 폴더에 있으며, CUDA 비주얼 스튜디오 통합도 마찬가지로 폴더에 있음.
- 각 디렉터리에는 설치 가능한 파일의 일부가 아니므로 무시할 수 있는 .dll 및 .nvi 파일이 있음.

1. Microsoft Windows용 CUDA 설치 가이드

- Conda를 사용하여 CUDA 소프트웨어 설치하기.

❖ Conda 설치 관리자를 사용할 때 CUDA의 설치 및 구성에 대해 설명.

❖ Conda 패키지는 <https://anaconda.org/nvidia> 에서 구할 수 있음.

❖ Conda Overview

❖ Conda 설치는 CUDA 툴킷을 설치 가능. 설치 단계는 다음과 같습니다.

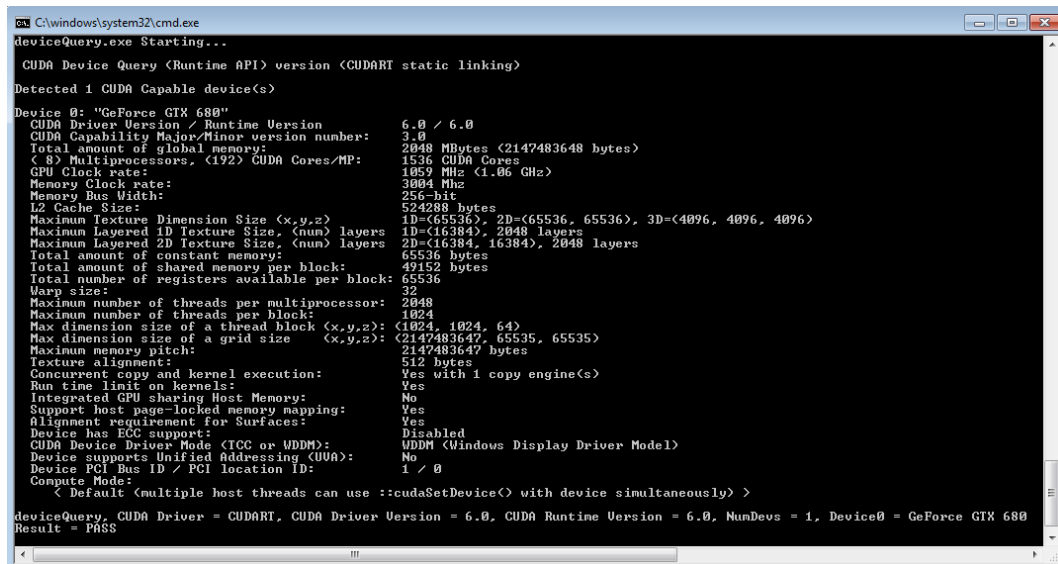
➤ 설치) `conda install cuda -c nvidia2.4.3.`

➤ 제거) `conda remove cuda`

1. Microsoft Windows용 CUDA 설치 가이드

■ 설치 확인

- ❖ CUDA 툴킷의 버전은 명령 프롬프트 창에서 `nvcc -V`를 실행하여 확인 가능
 - 시작 > 모든 프로그램 > 보조 프로그램 > 명령 프롬프트로 이동.
- ❖ CUDA 샘플은 <https://github.com/nvidia/cuda-samples> 에 있음.
- ❖ 샘플을 사용하려면 프로젝트를 복제하고 샘플을 빌드한 다음 Github 페이지의 지침에 따라 실행.
- ❖ 하드웨어 및 소프트웨어의 올바른 구성을 확인하려면 장치 쿼리 샘플 프로그램을 빌드하고 실행하는 것이 좋음.
- ❖ 샘플은 deviceQuery 폴더에 제공된 VS 솔루션 파일을 사용하여 빌드할 수 있음.
- ❖ 여기서는 기본 설치 디렉터리 구조를 사용했다고 가정.
- ❖ CUDA가 올바르게 설치 및 구성되었다면 출력은 그림 1과 비슷하게 보일 것입니다.

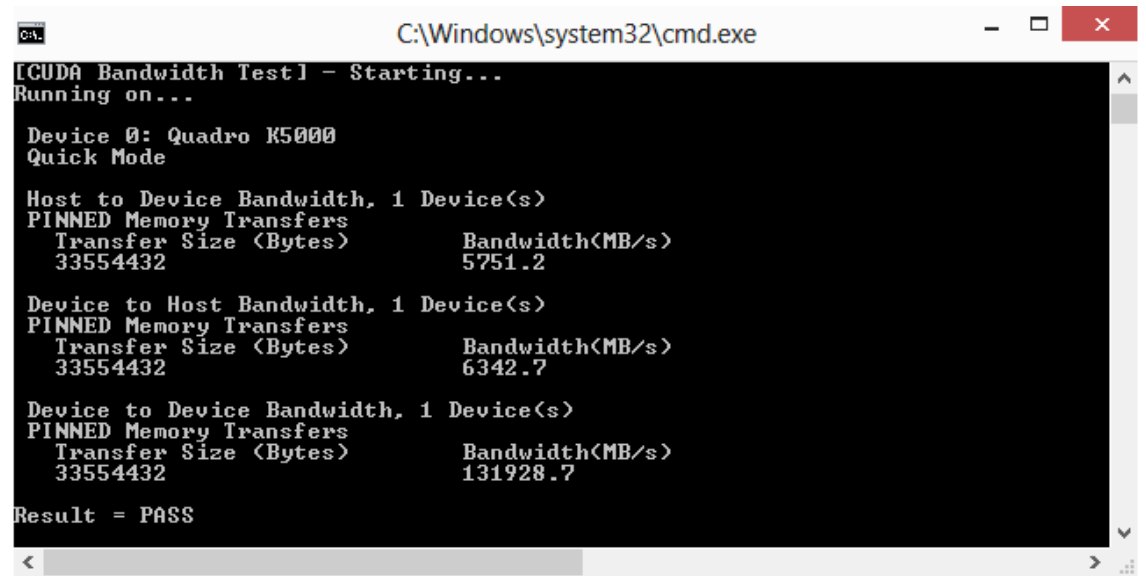


```
C:\windows\system32\cmd.exe
deviceQuery.exe Starting...

CUDA Device Query (Runtime API) version (CUDART static linking)
Detected 1 CUDA Capable device(s)

Device 0: "GeForce GTX 680"
CUDA Driver Version / Runtime Version      6.0 / 6.0
CUDA Capability Major/Minor version number: 3.0
Total amount of global memory:              2048 MBytes (2147483648 bytes)
( 8) Multiprocessors, (192) CUDA Cores/MP:  1536 CUDA Cores
GPU Clock rate:                             1057 Mhz (1.06 GHz)
Memory Clock rate:                          3004 Mhz
Memory Bus Width:                           256-bit
L2 Cache Size:                              524288 bytes
Maximum Texture Dimension Size (x,y,z)      1D=(65536, 2D=(65536, 65536), 3D=(4096, 4096, 4096)
Maximum Layered 1D Texture Size, (num) layers 1D=(16384), 2048 layers
Maximum Layered 2D Texture Size, (num) layers 2D=(16384, 16384), 2048 layers
Total amount of constant memory:             65536 bytes
Total amount of shared memory per block:     49152 bytes
Total number of registers available per block: 65536
Warp size:                                   32
Maximum number of threads per multiprocessor: 2048
Maximum number of threads per block:         1024
Max dimension size of a thread block (x,y,z): (1024, 1024, 64)
Max dimension size of a grid size    (x,y,z): (2147483647, 65535, 65535)
Maximum memory pitch:                       2147483647 bytes
Texture alignment:                           512 bytes
Concurrent copy and kernel execution:        Yes with 1 copy engine(s)
Run time limit on kernels:                   Yes
Integrated GPU sharing Host Memory:           No
Support host page-locked memory mapping:     Yes
Alignment requirement for Surfaces:          Yes
Device has ECC support:                      Disabled
CUDA Device Driver Mode (CIC or WDDM):        WDDM (Windows Display Driver Model)
Device supports Unified Addressing (UVA):     No
Device PCI Bus ID / PCI location ID:         1 / 0
Compute Mode:                                < Default (multiple host threads can use ::cudaSetDevice() with device simultaneously) >

deviceQuery, CUDA Driver = CUDART, CUDA Driver Version = 6.0, CUDA Runtime Version = 6.0, NumDevs = 1, Device0 = GeForce GTX 680
Result = PASS
```



```
C:\Windows\system32\cmd.exe

[CUDA Bandwidth Test] - Starting...
Running on...

Device 0: Quadro K5000
Quick Mode

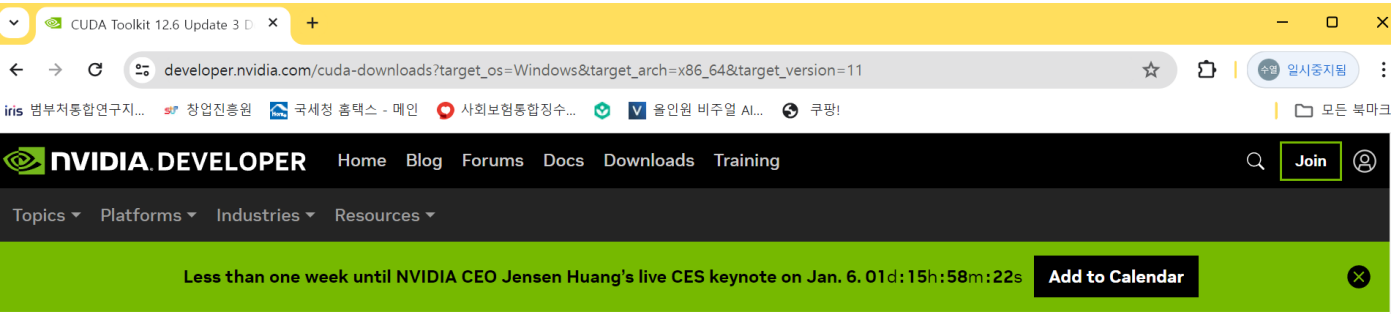
Host to Device Bandwidth, 1 Device(s)
PINNED Memory Transfers
Transfer Size (Bytes)      Bandwidth(MB/s)
33554432                   5751.2

Device to Host Bandwidth, 1 Device(s)
PINNED Memory Transfers
Transfer Size (Bytes)      Bandwidth(MB/s)
33554432                   6342.7

Device to Device Bandwidth, 1 Device(s)
PINNED Memory Transfers
Transfer Size (Bytes)      Bandwidth(MB/s)
33554432                   131928.7

Result = PASS
```

CUDA Toolkit 12.6 Update 3 Downloads



CUDA Toolkit 12.6 Update 3 Downloads

Select Target Platform

Click on the green buttons that describe your target platform. Only supported platforms will be shown. By downloading and using the software, you agree to fully comply with the terms and conditions of the [CUDA EULA](#).

Operating System	Linux Windows
Architecture	x86_64
Version	10 11 Server 2022
Installer Type	exe (local) exe (network)

Download Installer for Windows 11 x86_64

The base installer is available for download below.

> CUDA Toolkit Installer

Download (3.0 G

Installation Instructions:

1. Double click cuda_12.6.3_561.17_windows.exe
2. Follow on-screen prompts

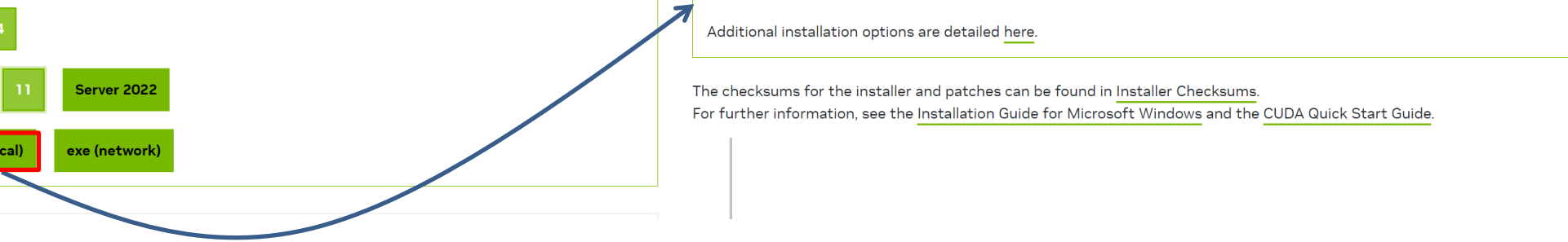
Additional installation options are detailed [here](#).

The checksums for the installer and patches can be found in [Installer Checksums](#).
For further information, see the [Installation Guide for Microsoft Windows](#) and the [CUDA Quick Start Guide](#).

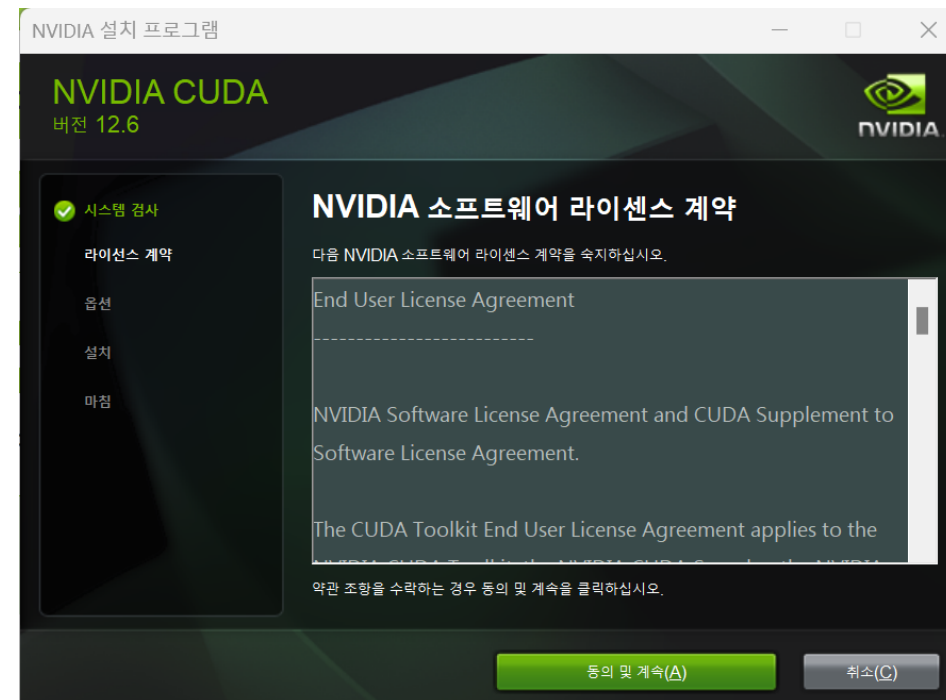
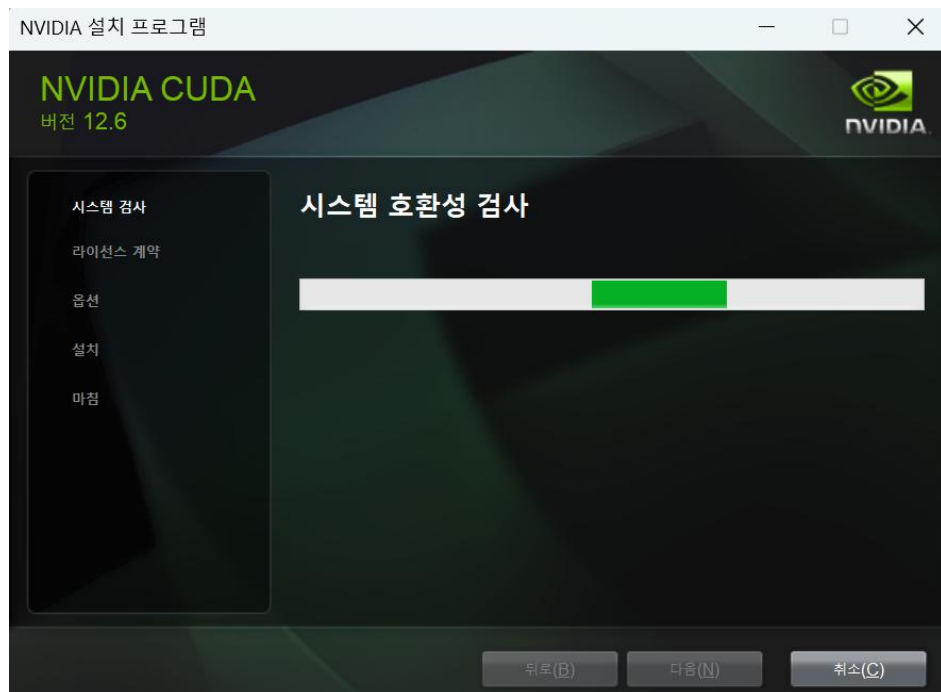
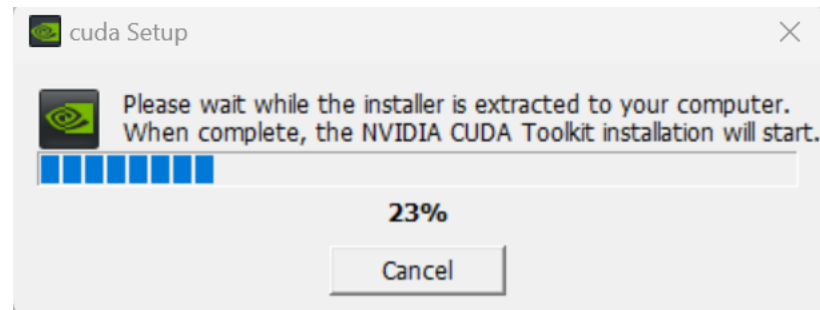
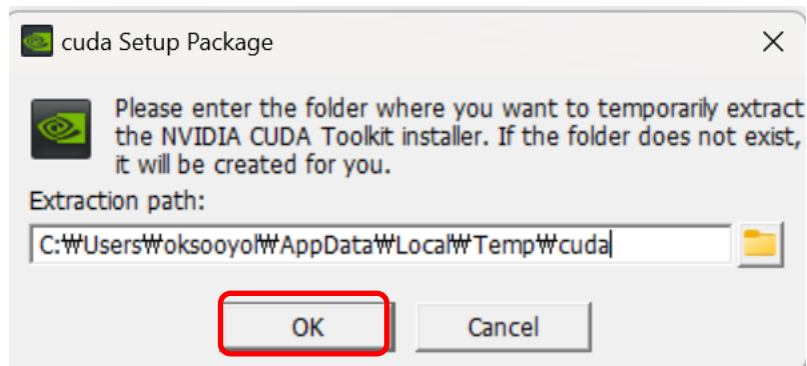
최근 다운로드 기록

cuda_12.6.3_561.17_windows.exe
↓ 2.7/3.0GB • 11초 남음

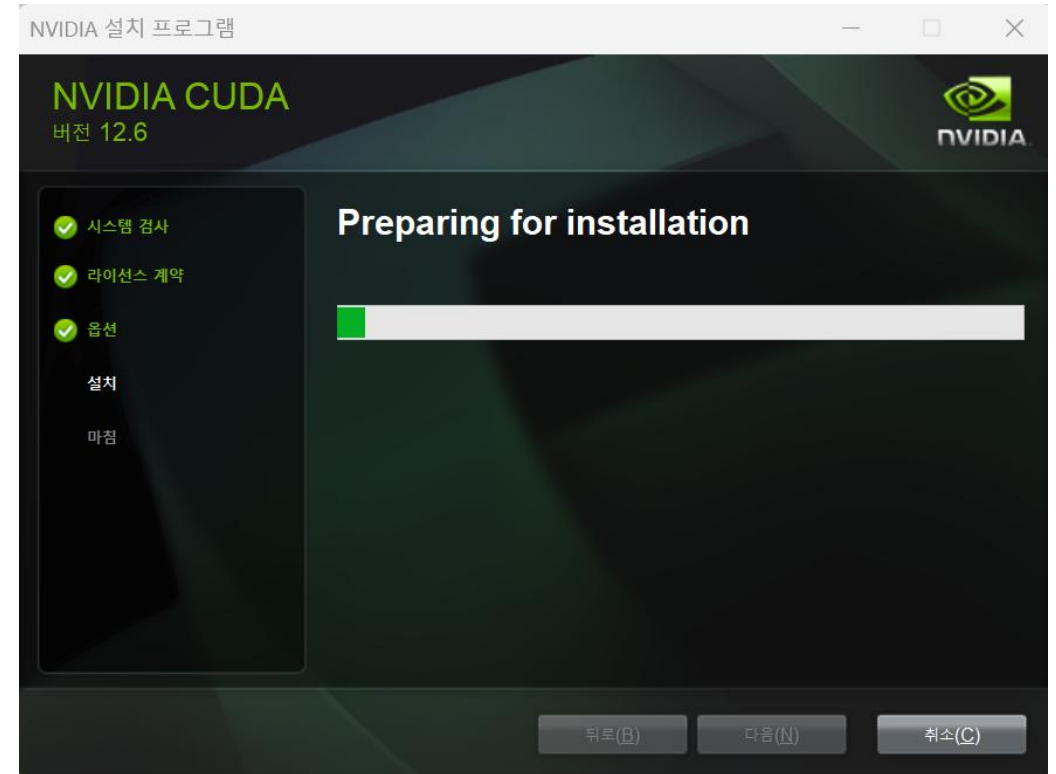
전체 다운로드 기록



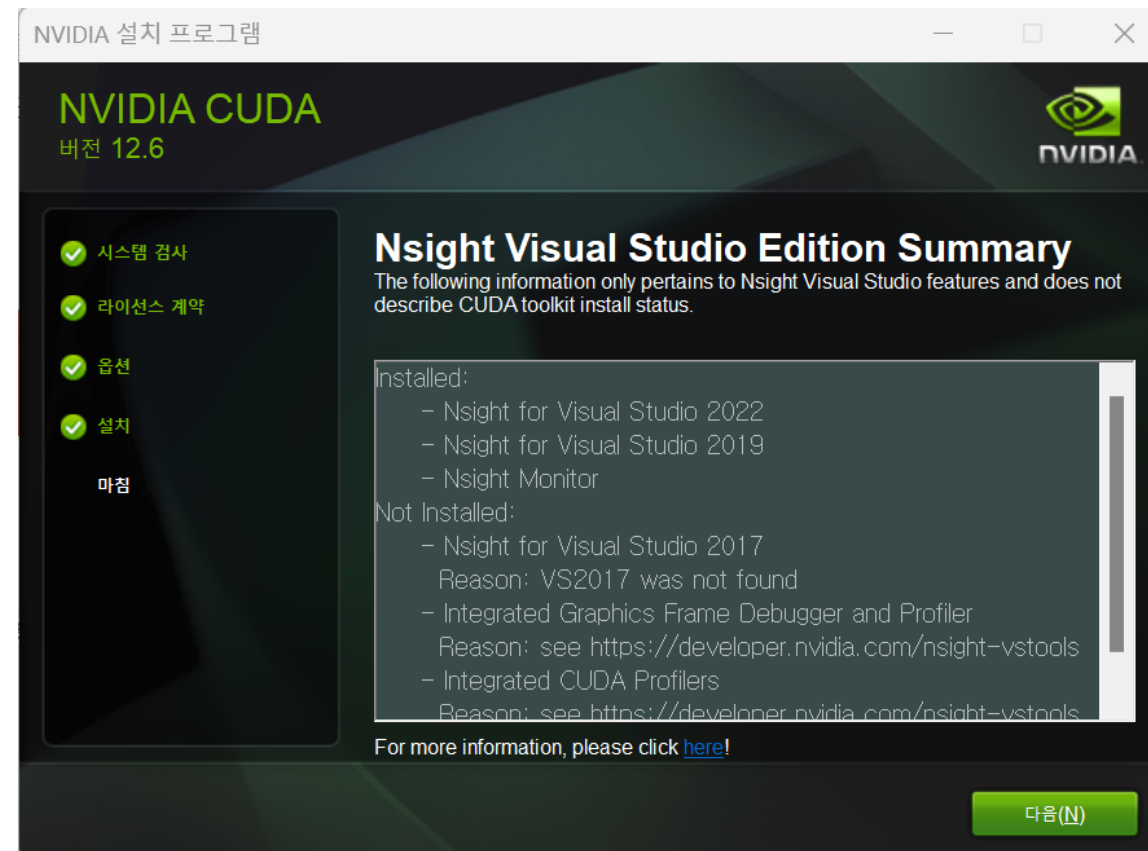
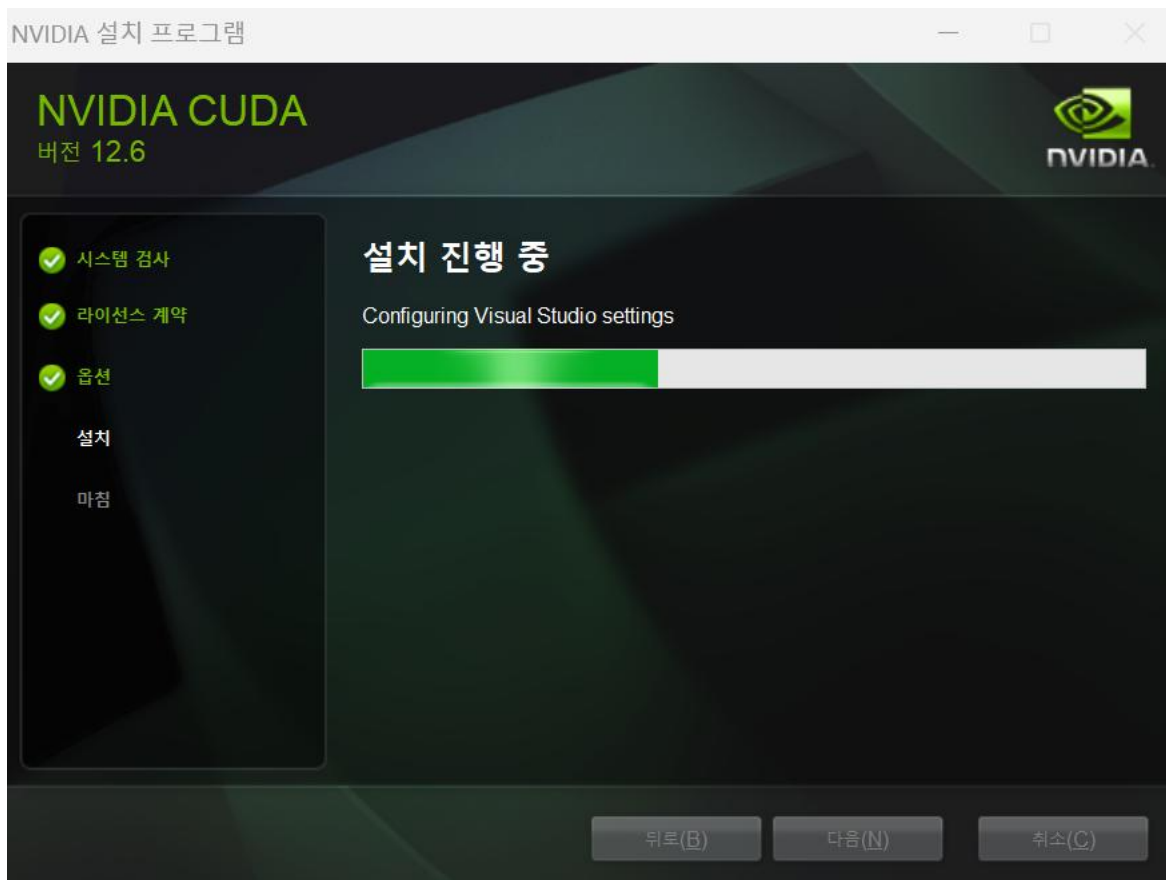
CUDA Toolkit 12.6 Update 3 Downloads



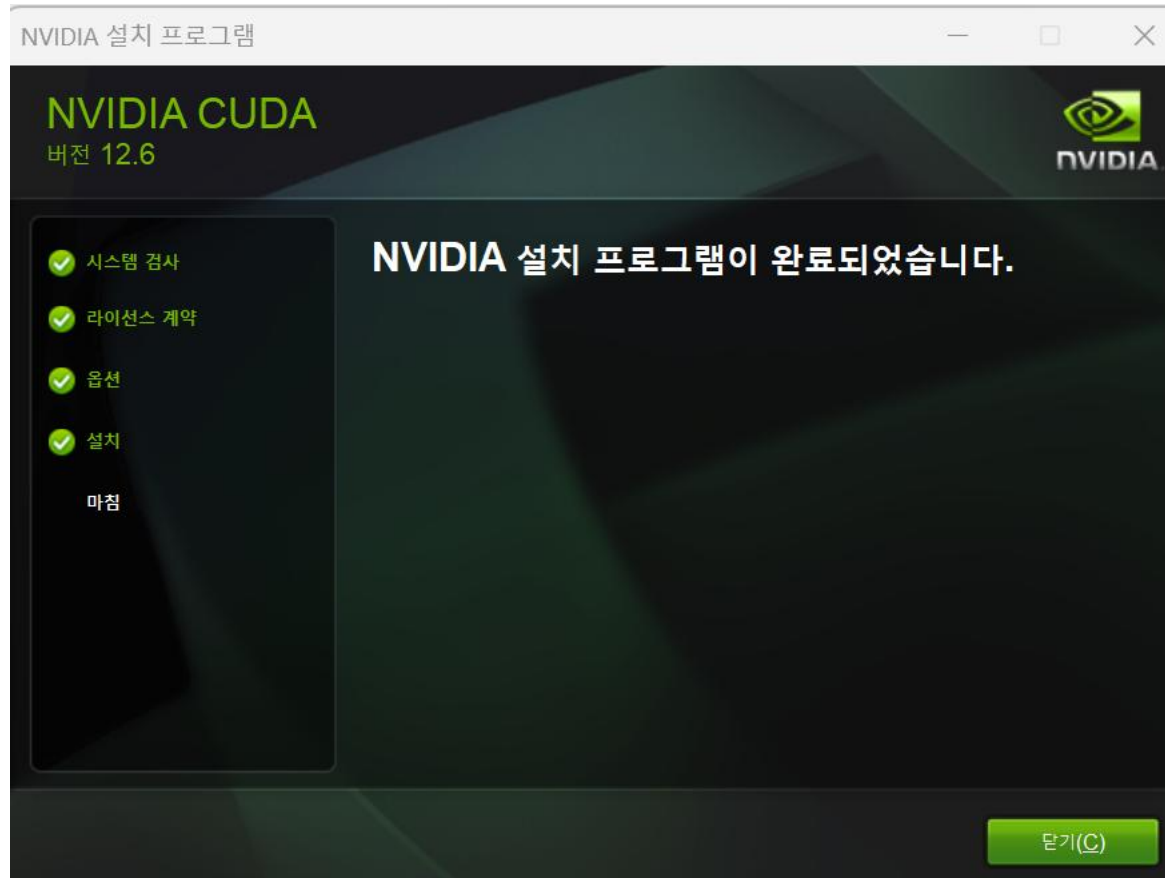
CUDA Toolkit 12.6 Update 3 Downloads



CUDA Toolkit 12.6 Update 3 Downloads



CUDA Toolkit 12.6 Update 3 Downloads



2. Linux (ubuntu 24.04) cuda toolkit 설치

❖ [How to Install CUDA on Ubuntu 24.04: Step-by-Step](#)

CUDA Parallel Computing Platform

www.nvidia.com/getcuda

Programming
Approaches

Libraries

“Drop-in” Acceleration

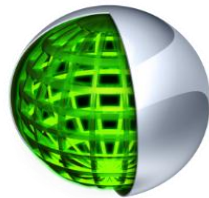
OpenACC Direct
ives

Easily Accelerate Apps

Programming L
anguages

Maximum Flexibility

Development
Environment



Nsight IDE
Linux, Mac and Windows
GPU Debugging and Profiling

CUDA-GDB debugger
NVIDIA Visual Profile
r

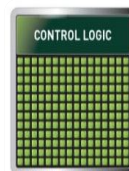
Open Compiler
Tool Chain



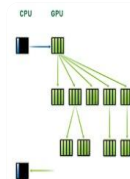
Enables compiling new languages to CUDA platform, and CU
DA languages to other architectures

Hardware
Capabilities

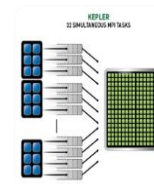
SMX



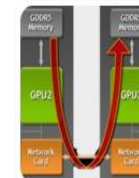
Dynamic Parallelism



HyperQ



GPUDirect

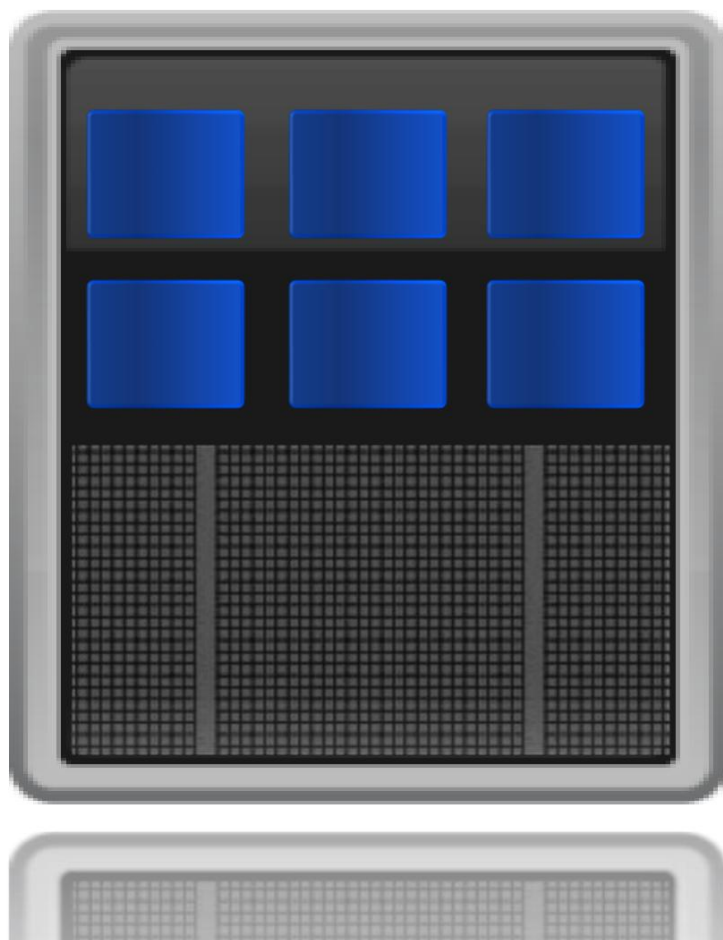


- <https://github.com/NVIDIA/cuda-samples>

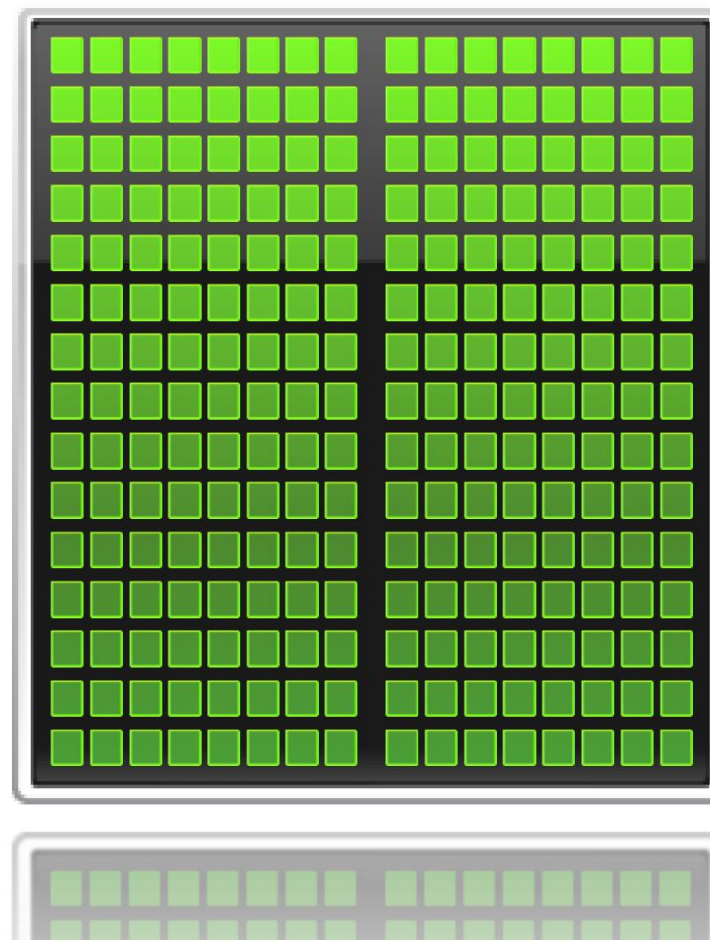
[NVIDIA/cuda-samples: Samples for CUDA Developers
which demonstrates features in CUDA Toolkit](https://github.com/NVIDIA/cuda-samples)

Add GPUs: Accelerate Science Applications

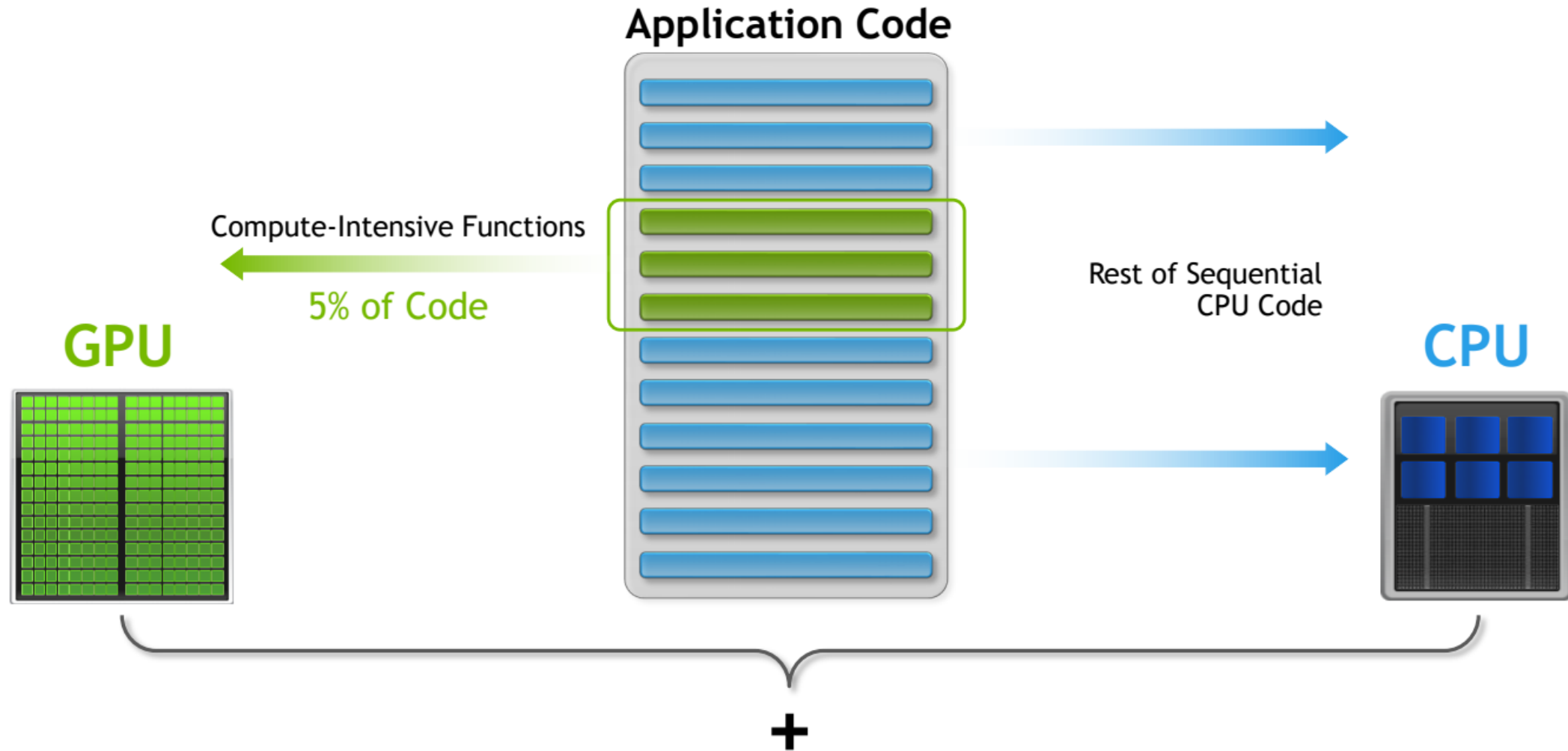
CPU



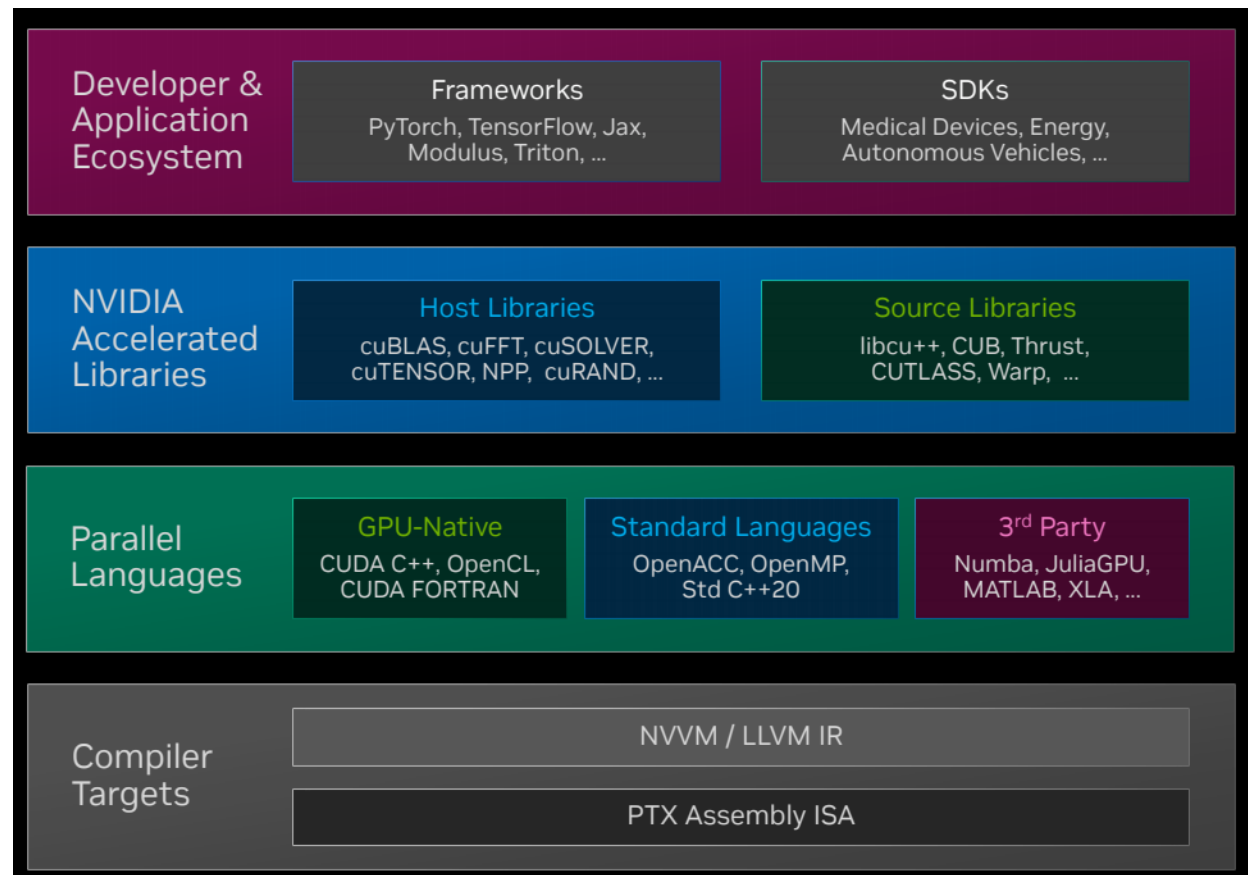
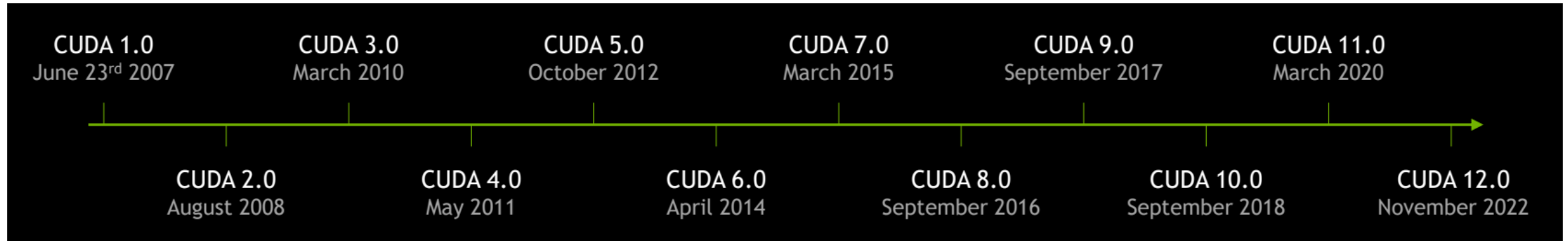
GPU



SMALL CHANGES, BIG SPEED-UP



CUDA



3 Ways to Accelerate Applications

Applications

Libraries

“Drop-in”
Acceleration

OpenACC
Directives

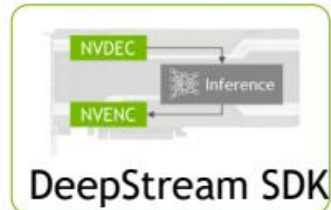
Easily Accelerate
Applications

Programming
Languages

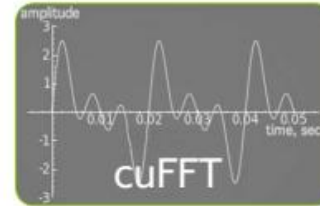
Maximum
Flexibility

GPU ACCELERATED LIBRARIES

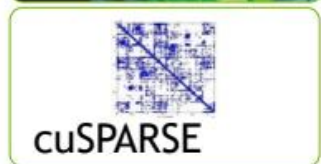
DEEP LEARNING



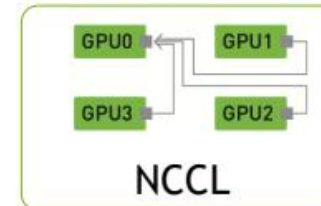
SIGNAL, IMAGE & VIDEO



LINEAR ALGEBRA



PARALLEL ALGORITHMS



STEPS TO CUDA-ACCELERATED APPLICATION

Step 1: Substitute library calls with equivalent CUDA library calls

```
saxpy ( ... ) ➤ cublasSaxpy ( ... )
```

Step 2: Manage data locality

- with CUDA: `cudaMalloc()`, `cudaMemcpy()`, etc.
- with CUBLAS: `cublasAlloc()`, `cublasSetVector()`, etc.

Step 3: Rebuild and link the CUDA-accelerated library

```
gcc myobj.o -l cublas
```

GPU Programming Languages

Numerical analytics ▶

MATLAB, Mathematica, LabVIEW

Fortran ▶

OpenACC, CUDA Fortran

C ▶

OpenACC, CUDA C

C++ ▶

Thrust, CUDA C++

Python ▶

PyCUDA, Copperhead

F# ▶

Alea.cuBase

Rapid Parallel C++ Development



- Resembles C++ STL
- High-level interface
 - Enhances developer productivity
 - Enables performance portability between GPUs and multiple CPUs
- Flexible
 - CUDA, OpenMP, and TBB backends
 - Extensible and customizable
 - Integrates with existing software
- Open source

```
// generate 32M random numbers on host
thrust::host_vector<int> h_vec(32 << 20);
thrust::generate(h_vec.begin(),
                 h_vec.end(),
                 rand);

// transfer data to device (GPU)
thrust::device_vector<int> d_vec = h_vec;

// sort data on device
thrust::sort(d_vec.begin(), d_vec.end());

// transfer data back to host
thrust::copy(d_vec.begin(),
             d_vec.end(),
             h_vec.begin());
```

<http://developer.nvidia.com/thrust> or <http://thrust.googlecode.com>

이 언어들은 모든 CUDA 지원 GPU에서 지원.

CUDA C/C++

<http://developer.nvidia.com/cuda-toolkit>

GPU.NET

<http://tidepowerd.com>

Thrust C++ Template Library

<http://developer.nvidia.com/thrust>

MATLAB

<http://www.mathworks.com/discovery/matlab-gpu.html>

CUDA Fortran

<http://developer.nvidia.com/cuda-toolkit>

Mathematica

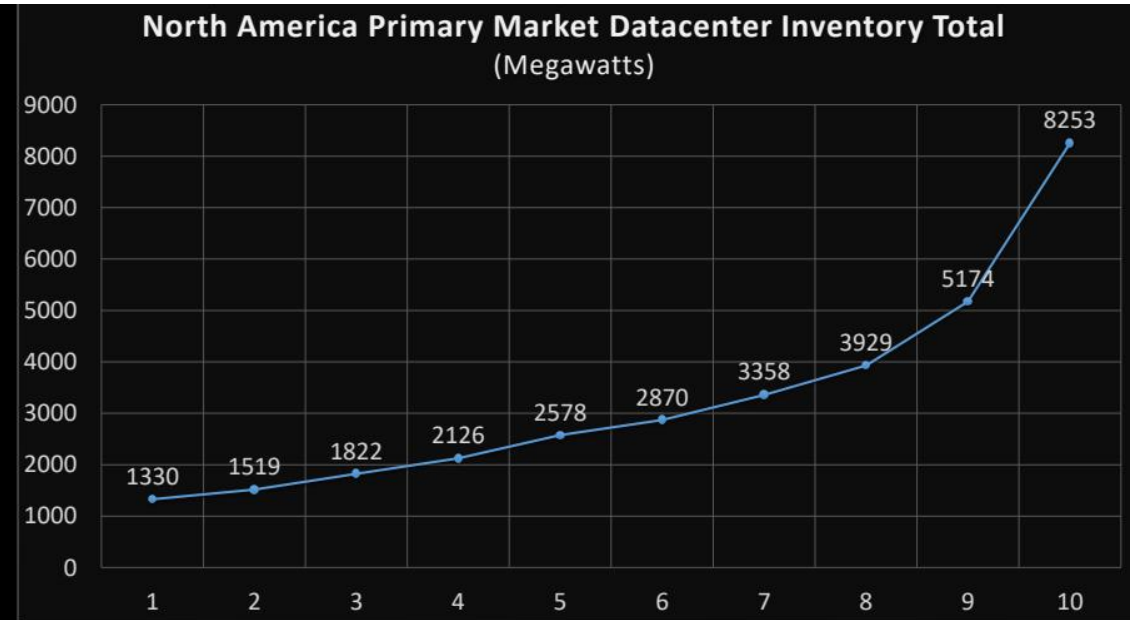
<http://www.wolfram.com/mathematica/new-in-8/cuda-and-opencl-support/>

PyCUDA (Python)

<http://mathematician.de/software/pycuda>

CUDA: New Features and Beyond(2024)

■ Accelerated Computing is not all about performance



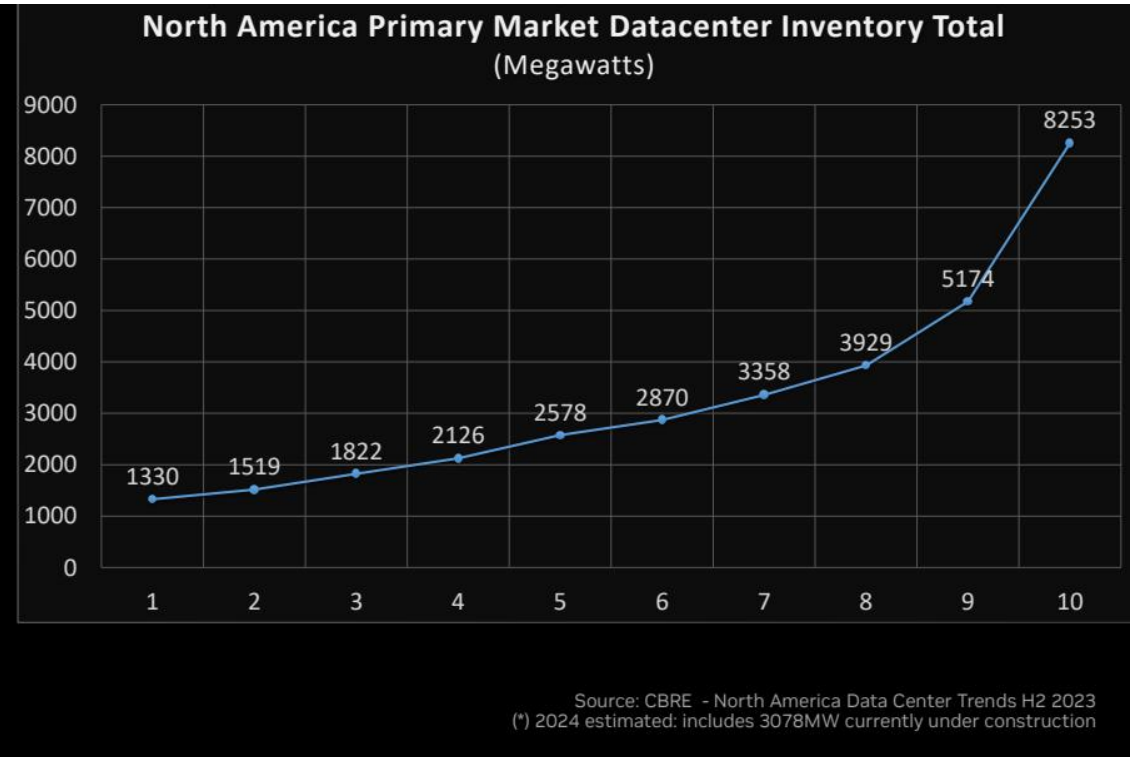
Source: CBRE - North America Data Center Trends H2 2023
(*) 2024 estimated: includes 3078MW currently under construction

데이터센터의 전력 소비 증가:

- 북미 데이터센터의 전력 용량이 기하급수적으로 증가하고 있으며, 이에 따라 전력 사용량이 급증
- 이는 데이터센터 운영 비용 증가를 의미하며, 에너지 효율성이 중요한 이유 중 하나.
- 예를 들어, 일반적인 중간 크기의 데이터센터(20,000 ~ 100,000 평방 피트, 2,000 ~ 10,000 서버)에서 최대 시스템 전력은 1.5kW에서 3kW까지 달할 수 있음.

CUDA: New Features and Beyond(2024)

■ Accelerated Computing is not all about performance

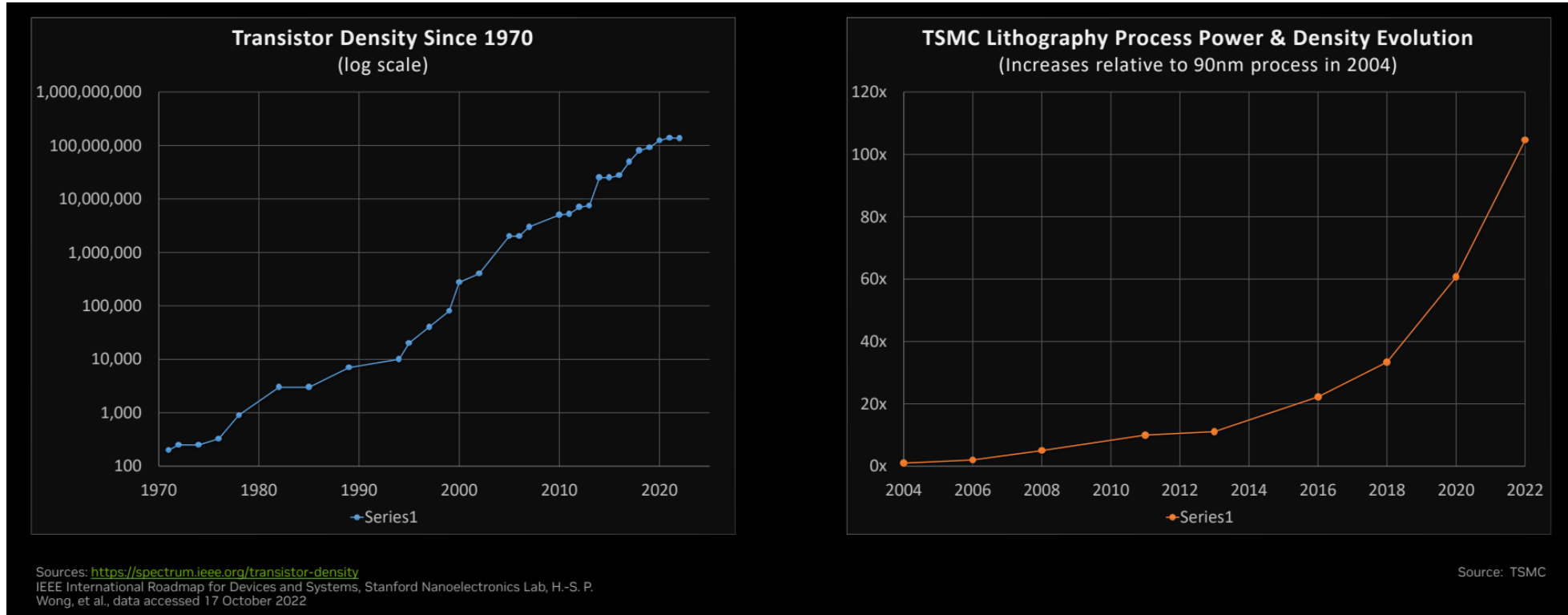


데이터센터의 전력 소비 증가:

- 북미 데이터센터의 전력 용량이 기하급수적으로 증가하고 있으며, 이에 따라 전력 사용량이 급증
- 이는 데이터센터 운영 비용 증가를 의미하며, 에너지 효율성이 중요한 이유 중 하나.
- 예를 들어, 일반적인 중간 크기의 데이터센터(20,000 ~ 100,000 평방 피트, 2,000 ~ 10,000 서버)에서 최대 시스템 전력은 1.5kW에서 3kW까지 달할 수 있음.

CUDA: New Features and Beyond(2024)

■ The Problem is That Transistor Density Keeps Increasing Exponentially



트랜지스터 밀도의 증가와 전력 효율성 감소:

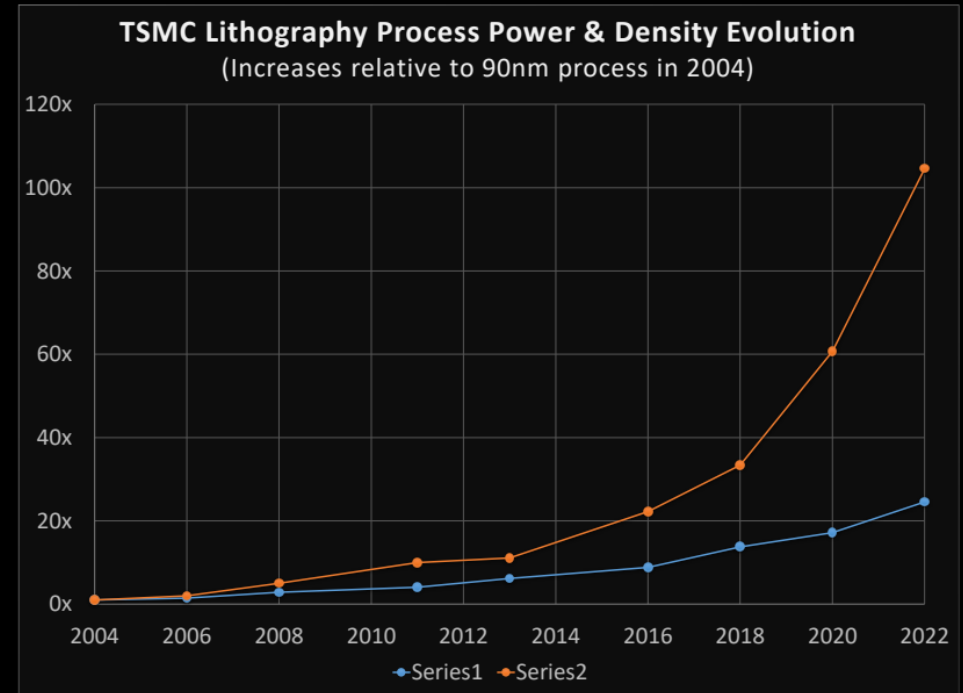
- 트랜지스터 밀도는 계속해서 증가하고 있지만, 트랜지스터의 전력 효율성은 감소.
- 트랜지스터가 더 많이 배치되더라도 동일한 전력으로 더 많은 성능을 제공하지는 못함.
- 따라서 전력 효율성이 중요한 요소로 부각.

CUDA: New Features and Beyond(2024)

■ 데이터 이동 및 계산에 필요한 전력 소비

We spend power on two things

1. Data movement
2. Computation



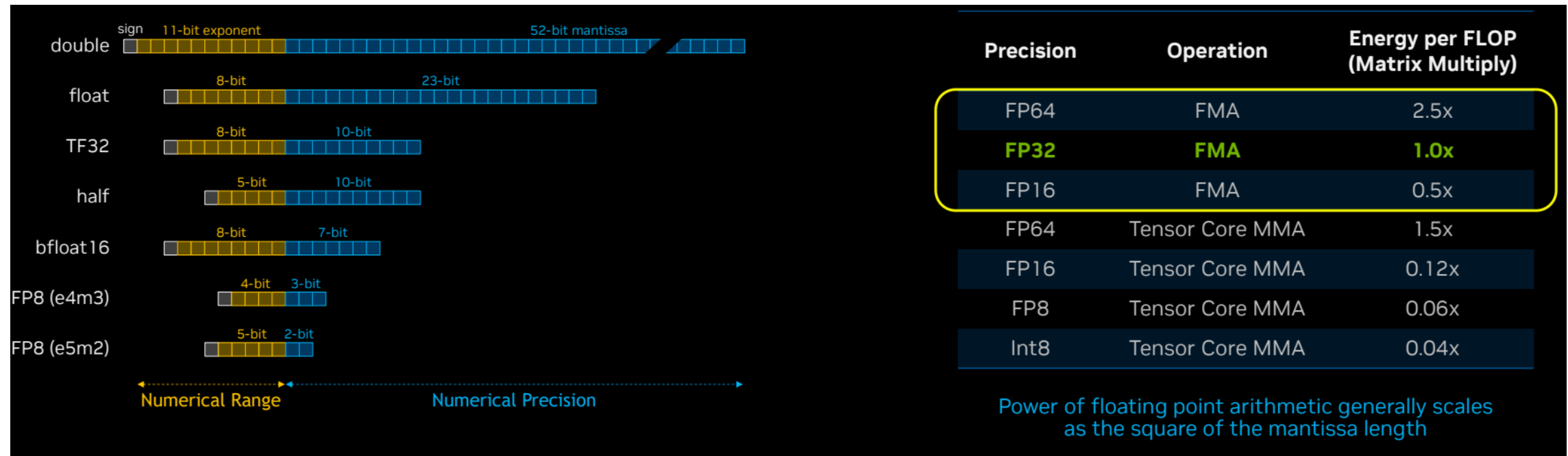
Source: TSMC

데이터 이동 및 계산에 필요한 전력 소비

- 시스템이 전력을 소모하는 주요 두 가지 활동은 데이터 이동과 계산.
- 특히 데이터 이동은 많은 전력을 소비하게 되며, 이는 시스템 전체의 에너지 효율성에 영향을 미침

CUDA: New Features and Beyond(2024)

■ 부동 소수점 연산의 에너지 비용

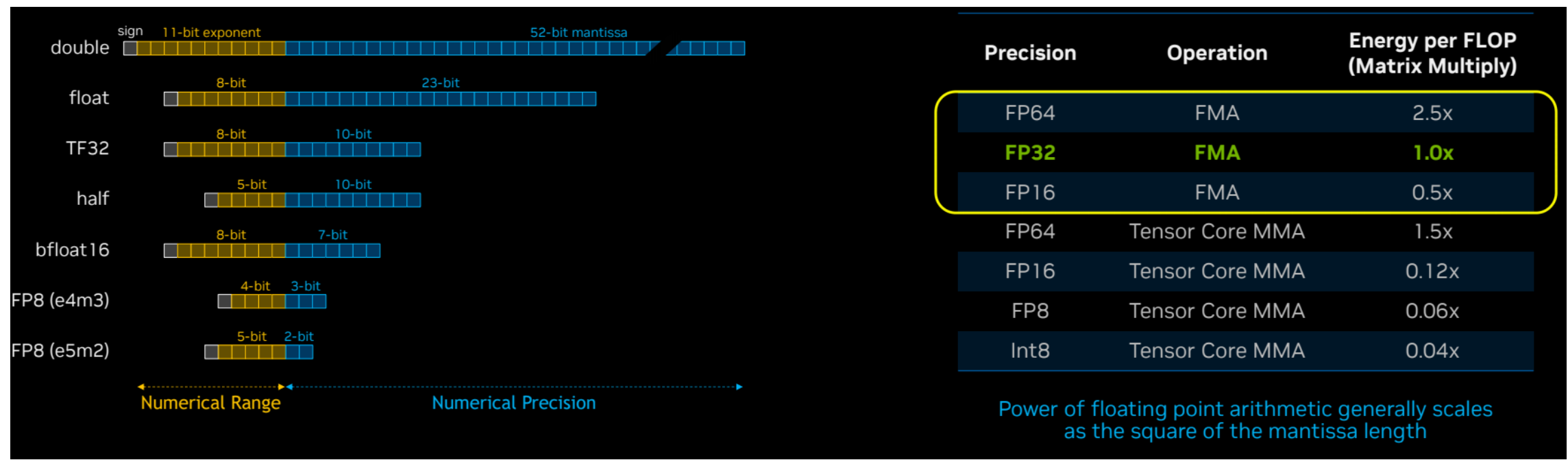


데이터 이동 및 계산에 필요한 전력 소비

- 부동 소수점 연산의 전력 비용은 연산의 정밀도에 따라 다름.
- 예를 들어, FP64(64비트 부동 소수점) 연산은 더 많은 전력을 소비하고, FP8(8비트 부동 소수점) 연산은 훨씬 적은 전력을 소비.
- Tensor Core를 사용한 연산은 일반적인 부동 소수점 연산보다 전력 효율성이 1.5배에서 4배 더 높음.

CUDA: New Features and Beyond(2024)

■ 부동 소수점 연산의 에너지 비용

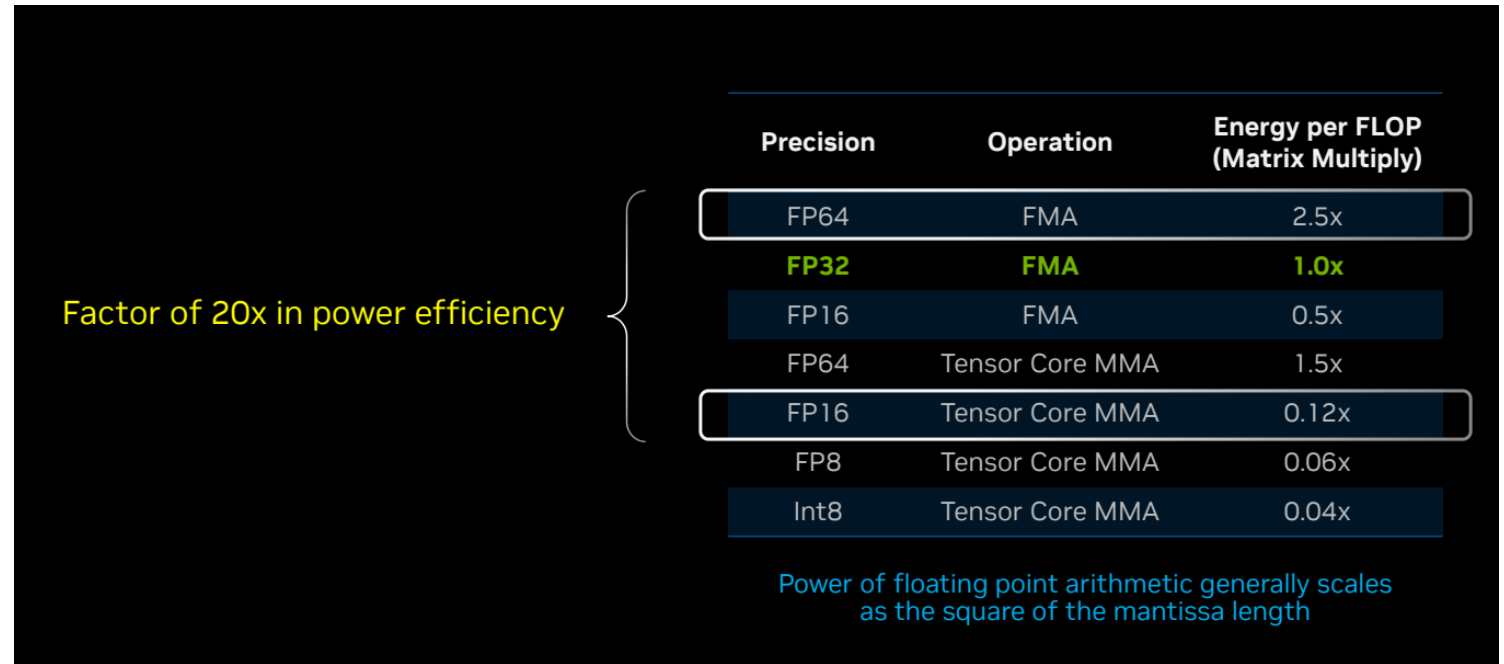


데이터 이동 및 계산에 필요한 전력 소비

- 부동 소수점 연산의 전력 비용은 연산의 정밀도에 따라 다름.
- 예를 들어, FP64(64비트 부동 소수점) 연산은 더 많은 전력을 소비하고, FP8(8비트 부동 소수점) 연산은 훨씬 적은 전력을 소비.
- Tensor Core를 사용한 연산은 일반적인 부동 소수점 연산보다 전력 효율성이 1.5배에서 4배 더 높음.

CUDA: New Features and Beyond(2024)

■ Tensor Core의 에너지 효율성

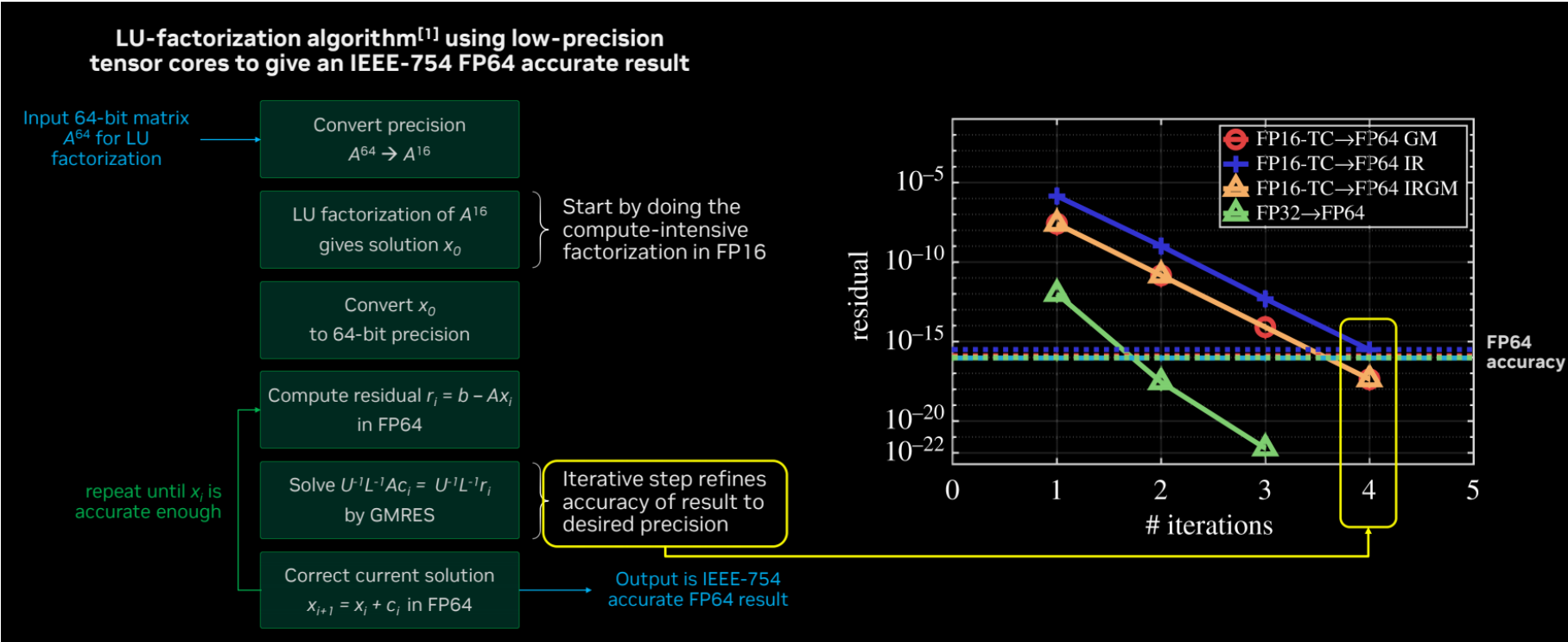


- Tensor Core는 일반적인 부동 소수점 연산보다 훨씬 더 높은 에너지 효율성을 제공.
- 예를 들어, FP64 부동 소수점 연산을 수행하는 데 소요되는 에너지를 기준으로, FP64 FMA(유동 소수점 연산)는 2.5배의 에너지를 소모하는 반면, FP64 Tensor Core MMA(매트릭스 매니플레이션 어큐뮬레이터) 연산은 1.5배의 에너지를 소모.
- FP16 및 FP8의 경우: Tensor Core를 사용할 때, FP16 및 FP8 정밀도의 연산은 더욱 효율적. FP16 Tensor Core MMA는 일반적인 FP16 FMA보다 0.12배의 에너지를 소모하며, FP8 Tensor Core MMA는 0.06배의 에너지를 소모. 이는 매우 높은 전력 효율성을 의미.

CUDA: New Features and Beyond(2024)

■ Solving High Energy Problems With Low Energy Transistor("Mixed-Precision Iterative Refinement"을 사용한 LU-분해 알고리즘의 효율성)

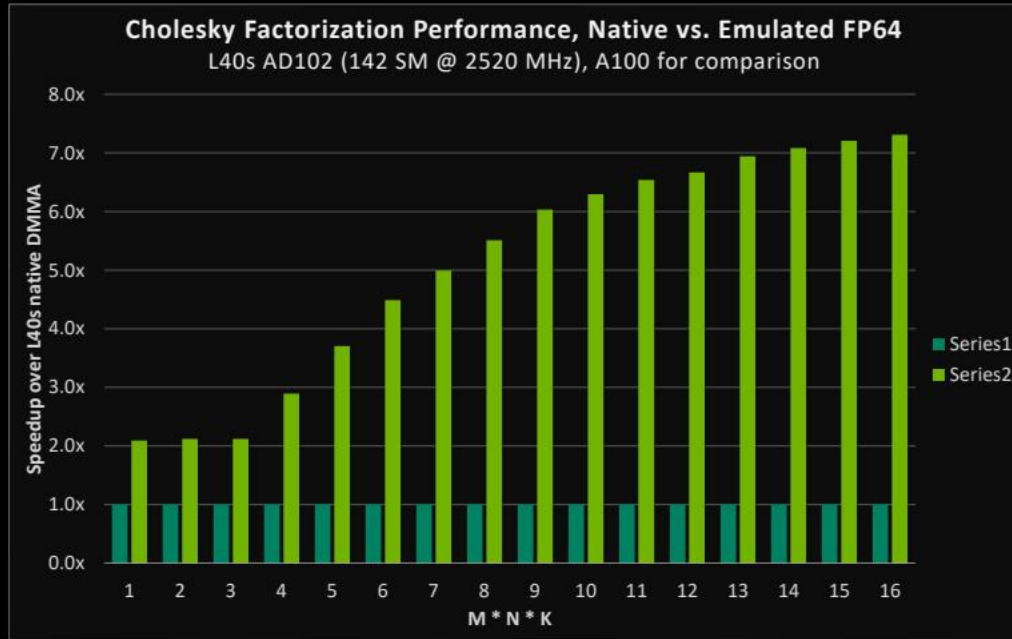
- 입력 및 변환: 64비트 정밀도의 행렬 A_{64} 를 16비트 정밀도의 행렬 A_{16} 로 변환. 이는 초기 LU-분해를 위한 단계
- LU 분해 및 초기 해 구하기: A_{16} 에서 LU-분해를 수행하여 초기 해 x_0 를 구함
- 정밀도 변환: 초기 해 x_0 를 64비트 정밀도로 변환.
- 잔차 계산: 64비트 정밀도로 잔차 $r_i = b - Ax_i$ 를 계산. 이는 현재 해 x_i 와 실제 값 b 사이의 차이를 나타냄.
- 보정 계산: GMRES 알고리즘을 사용하여 $U^{\{-1\}}L^{\{-1\}}Ac_i = U^{\{-1\}}L^{\{-1\}}r_i$ 를 해결함. 이를 통해 x_i 의 현재 해를 보정. 업데이트된 해는 $x_{i+1} = x_i + c_i$ 가 됨
- 반복: 위 과정을 반복하여 최종적으로 64비트 정밀도의 정확한 해 x 를 얻음



- 성능과 전력 효율성: **Tensor Core**를 사용하여 **FP16** 정밀도에서 연산을 수행함으로써 높은 성능과 전력 효율성을 유지할 수 있음.
- **GH200 GPU**를 사용한 복잡한 행렬 **LU-분해**의 경우, **FP64** 연산보다 약 **3.9배** 높은 성능과 **5.7배** 높은 와트당 성능을 기록함.

CUDA: New Features and Beyond(2024)

■ High Performance FP64 GEMM On Non-Native Hardware



Based on the method of Ootomo et. al.^[2] to apply integer tensor cores to DGEMM, L40s achieves a 7x performance increase over native FP64 (and reaches 54% of A100 FP64 tensor core performance)



NVIDIA L40S GPU

GPU Architecture	NVIDIA Ada Lovelace
GPU Memory	48GB GDDR6 w/ ECC
Memory Bandwidth	864 GB/s
FP64 TFLOPS	1.4
FP32 TFLOPS	91.6
TF32 Tensor Core FLOPS	183
Bfloat16 Tensor Core TFLOPS	362
FP16 Tensor Core TFLOPS	362
FP8 Tensor Core TFLOPS	733

- 최적화 기법:비네이티브 하드웨어에서 FP64 성능을 극대화하기 위해 Ootomo et al.의 방법을 적용,
- 이는 정수 텐서 코어를 DGEMM(Double-precision General Matrix Multiply)에 적용하는 것.
- 이를 통해 L40S GPU는 네이티브 FP64 대비 약 7배의 성능을 달성, A100 GPU의 FP64 텐서 코어 성능의 절반 이상을 달성함.

CUDA: New Features and Beyond(2024)

■ Tensor Core Programming Abstractions



NVIDIA의 다양한 라이브러리와 API를 통한 Tensor Core를 효과적인 활용 방법

CUDA: New Features and Beyond(2024)

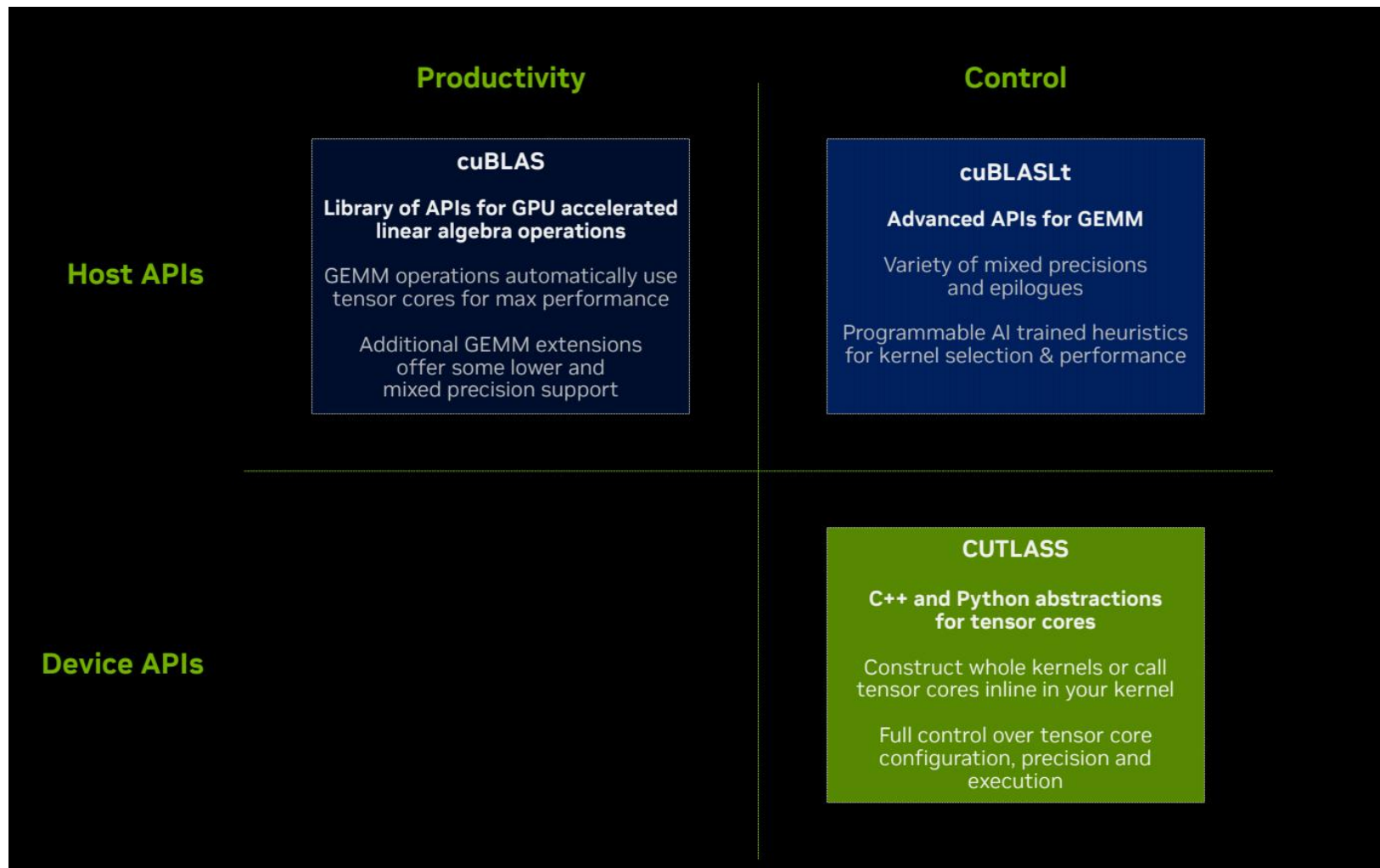
■ Tensor Core Programming Abstractions



NVIDIA의 다양한 라이브러리와 API를 통한 Tensor Core를 효과적인 활용 방법

CUDA: New Features and Beyond(2024)

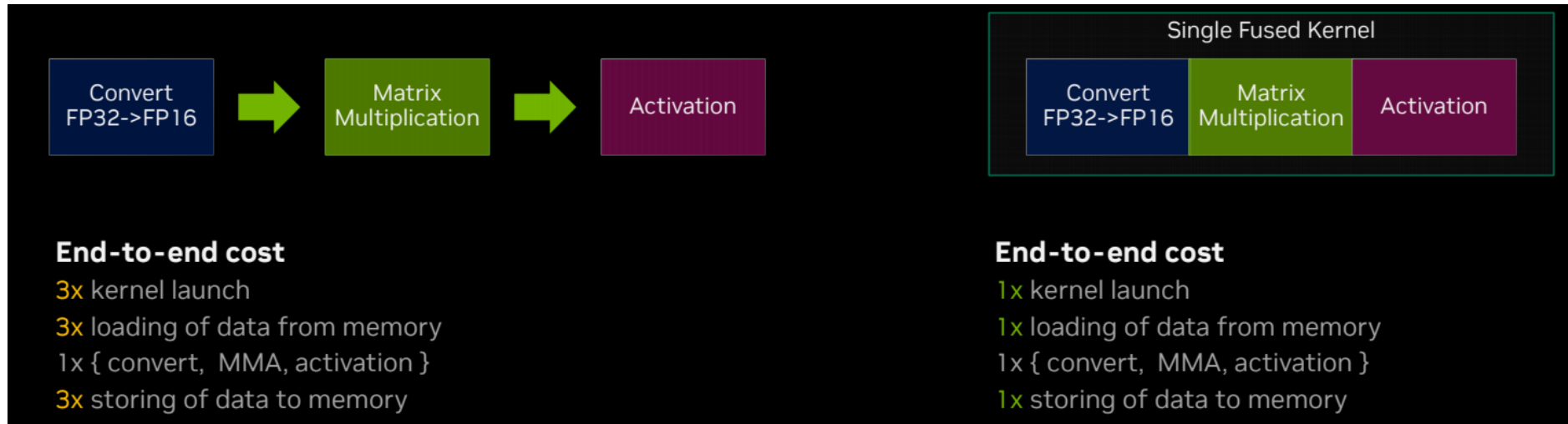
■ Tensor Core Programming Abstractions



CUDA: New Features and Beyond(2024)

■ Basics of Kernel Fusion

여러 개의 커널 실행을 하나의 커널 실행으로 결합하여 성능을 최적화하고 에너지 효율성을 높이는 방법



단일 퓨전 커널:

- 여러 개의 연산을 하나의 커널로 결합하여 실행.
- 예를 들어, FP32에서 FP16으로 변환, 행렬 곱셈, 그리고 활성화 연산을 하나의 퓨전 커널로 결합
- 이를 통해 개별 커널 실행보다 더 높은 성능과 에너지 효율성을 달성할 수 있음.

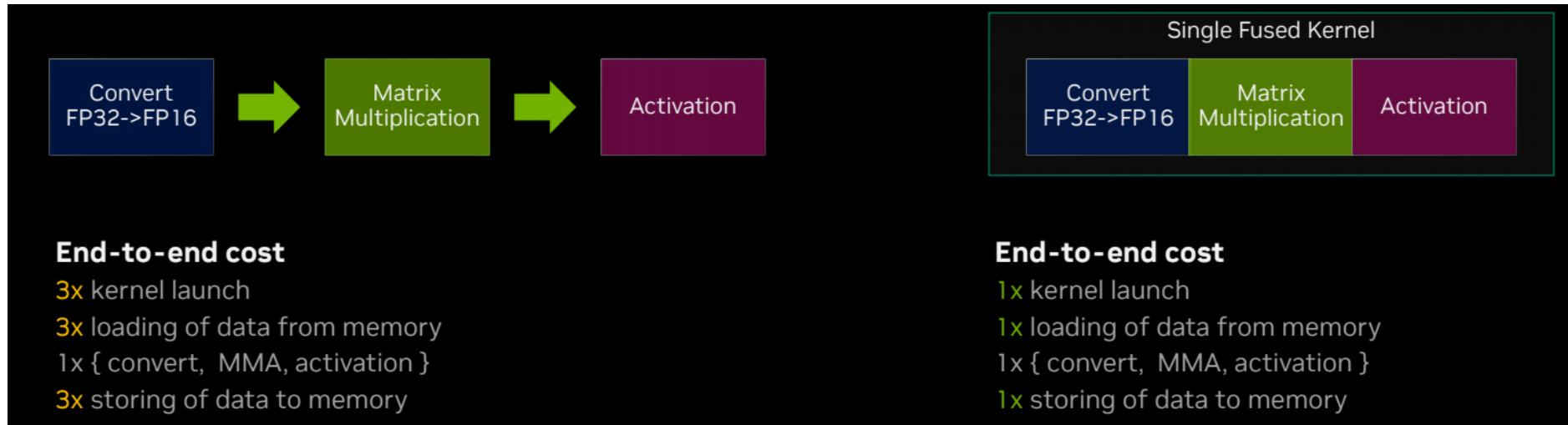
커널 실행 비용 절감:

- 전통적인 방법으로 각 연산을 개별 커널로 실행하면 3번의 커널 실행, 3번의 메모리로부터 데이터 로드, 3번의 데이터 저장이 필요.
- 반면, 단일 퓨전 커널을 사용하면 한 번의 커널 실행, 한 번의 메모리 로드, 한 번의 데이터 저장만 필요

CUDA: New Features and Beyond(2024)

■ Kernel Fusion Can Lead To Specialization Explosion

여러 개의 커널 실행을 하나의 커널 실행으로 결합하여 성능을 최적화하고 에너지 효율성을 높이는 방법



단일 퓨전 커널:

- 여러 개의 연산을 하나의 커널로 결합하여 실행.
- 예를 들어, FP32에서 FP16으로 변환, 행렬 곱셈, 그리고 활성화 연산을 하나의 퓨전 커널로 결합
- 이를 통해 개별 커널 실행보다 더 높은 성능과 에너지 효율성을 달성할 수 있음.

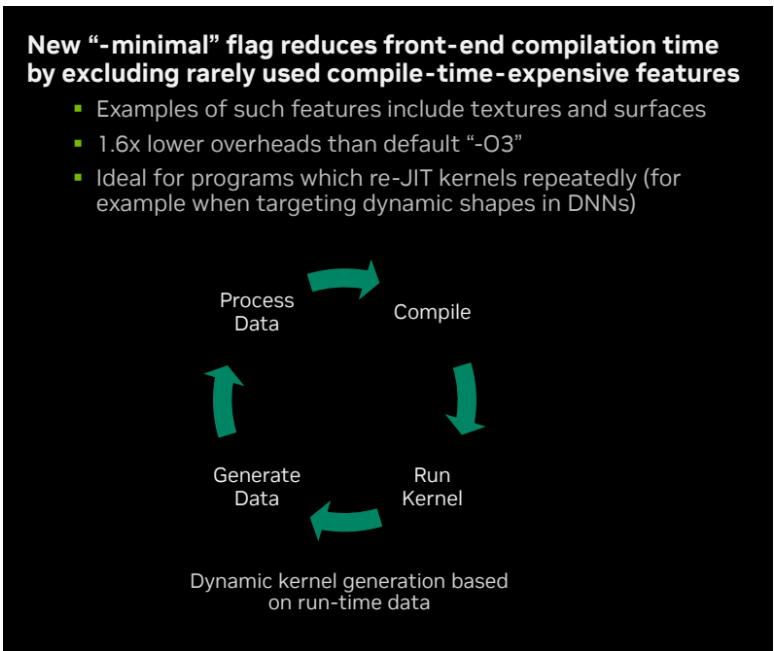
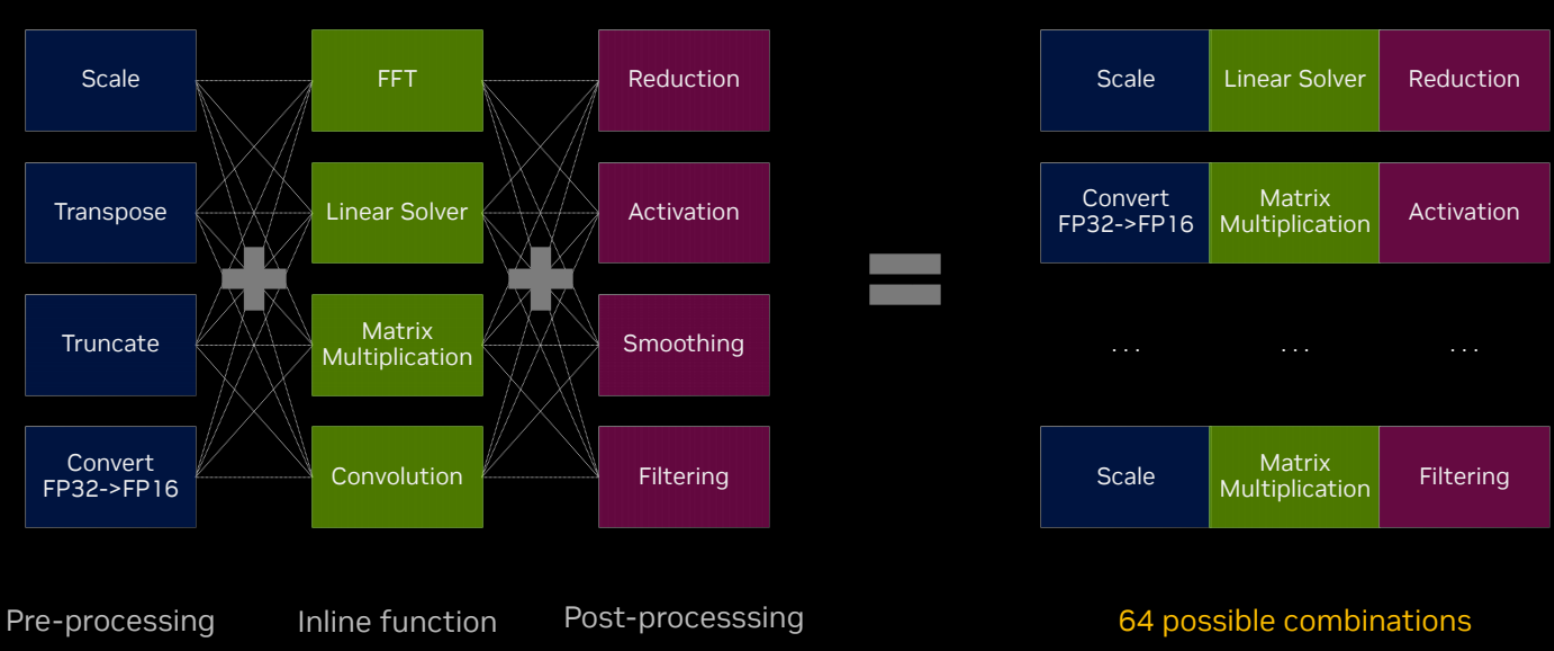
커널 실행 비용 절감:

- 전통적인 방법으로 각 연산을 개별 커널로 실행하면 3번의 커널 실행, 3번의 메모리로부터 데이터 로드, 3번의 데이터 저장이 필요.
- 반면, 단일 퓨전 커널을 사용하면 한 번의 커널 실행, 한 번의 메모리 로드, 한 번의 데이터 저장만 필요

CUDA: New Features and Beyond(2024)

Kernel Fusion Can Lead To Specialization Explosion

- 다양한 연산의 조합으로 인해 많은 수의 전문화된 커널이 필요할 수 있습니다. 이는 유지 보수와 관리가 어렵게 만듦



동적 퓨전 및 JIT 컴파일단일 퓨전 커널:

- 런타임에 필요한 퓨전을 동적으로 생성하고, Just-In-Time (JIT) 컴파일을 사용하여 이러한 문제를
- 이를 통해 다양한 연산 조합에 대해 효율적으로 대응가능.

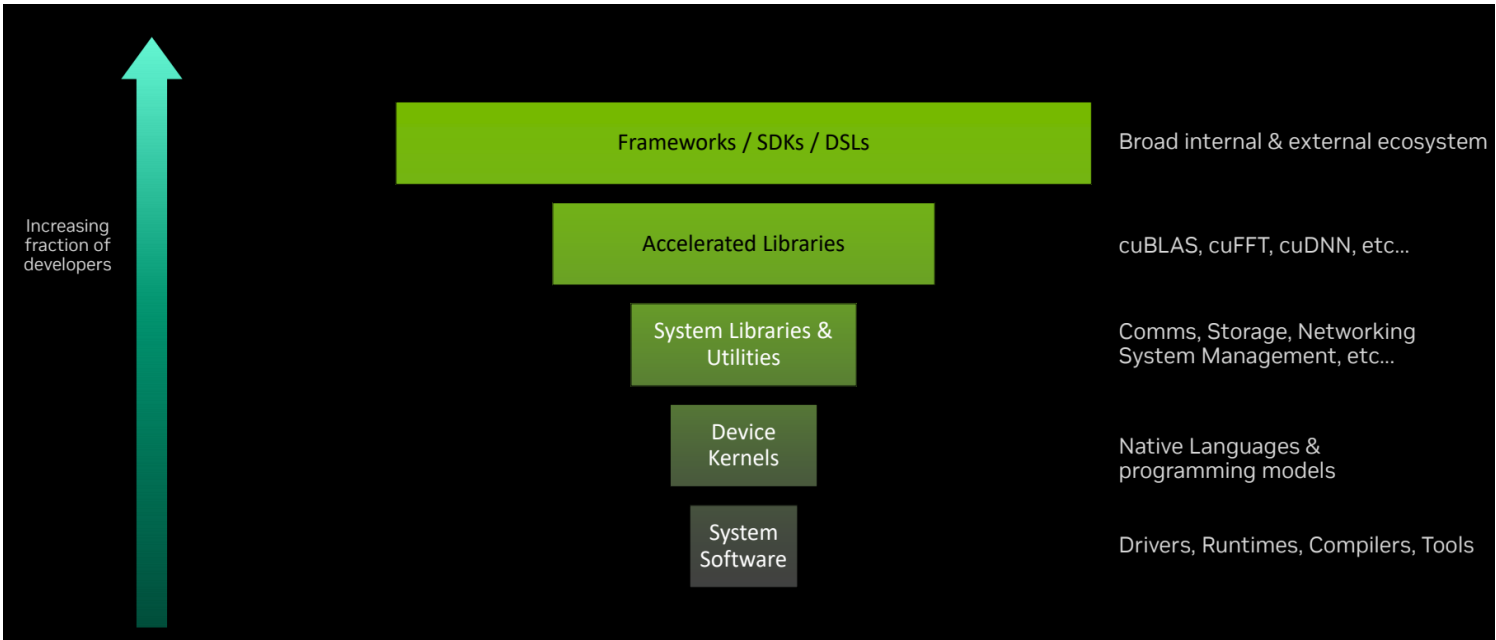
예시

- 행렬 곱셈과 활성화 연산, 변환 연산 등을 결합하여 단일 커널로 실행하면, 메모리 접근 횟수를 줄이고 성능을 크게 향상시킬 수 있음.
- 다이어그램에서 보여주는 것처럼, 퓨전된 커널은 여러 번의 커널 실행을 하나로 합쳐서 성능을 최적화함

CUDA: New Features and Beyond(2024)

■ The Inverted CUDA Developer Pyramid()

▪ 저수준 함수(Low-level functions)의 중요성



프레임워크/SDK/DSL: 개발자가 CUDA 생태계를 쉽게 활용할 수 있도록 돕는 다양한 프레임워크, 소프트웨어 개발 키트(SDK), 도메인 특화 언어(DSL) 등이 포함됨. 예를 들어, TensorFlow, PyTorch 등의 프레임워크가 있음.

가속 라이브러리: 고성능 연산을 위해 최적화된 라이브러리. 예를 들어, 선형 대수 연산을 가속화하는 cuBLAS, 신경망 연산을 가속화하는 cuDNN 등이 있음.

시스템 라이브러리 및 유틸리티: 통신, 스토리지, 네트워킹, 시스템 관리 등의 기능을 제공하는 라이브러리와 유틸리티가 포함됨. 예를 들어, cuBLAS, cuFFT, cuDNN 등이 있음. 프레임워크가 있음.

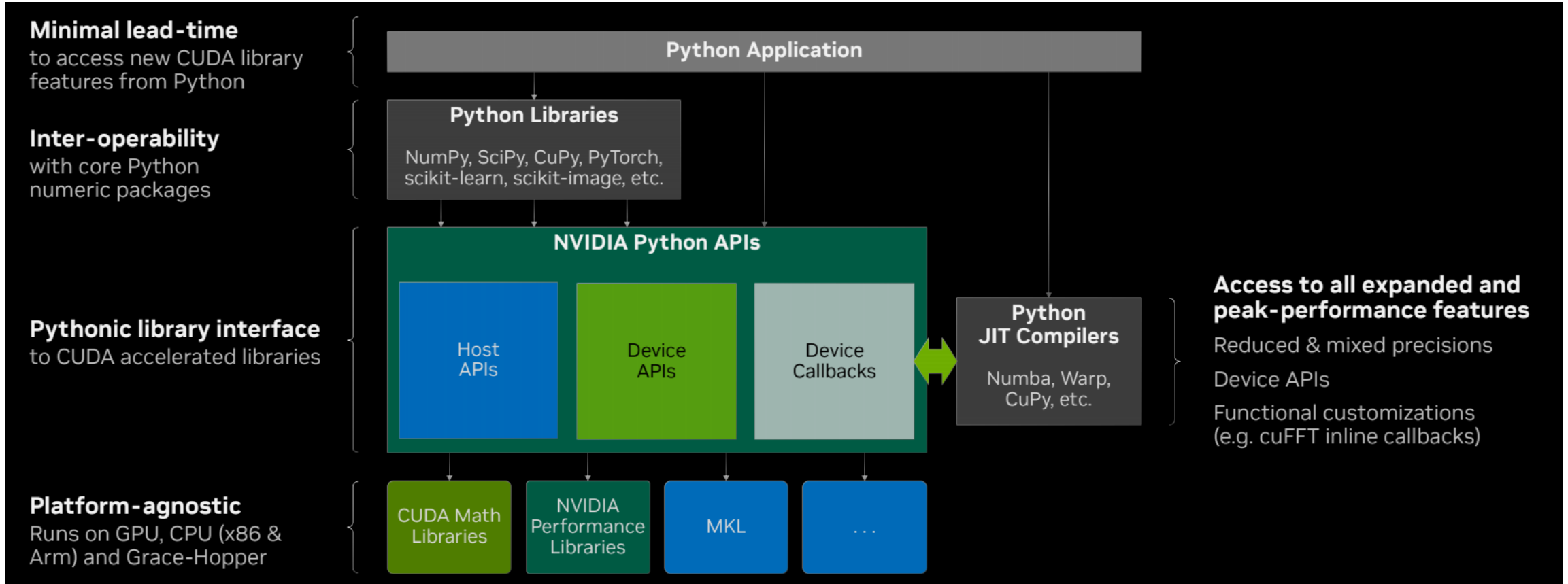
장치 커널: GPU에서 실행되는 커널 코드로, CUDA 프로그램의 핵심 부분. 장치 커널은 고성능 컴퓨팅을 위해 최적화되어 있음. PyTorch 등의 프레임워크가 있음.

시스템 소프트웨어: 드라이버, 런타임, 컴파일러, 도구 등이 포함됨. 이는 저수준 함수로 구성되어 있으며, 하드웨어와 직접 상호작용함. 있음.

내부 및 외부 생태계: 이 피라미드는 내부 및 외부 개발자 생태계를 포함하여 다양한 수준의 개발자들이 CUDA 생태계를 활용할 수 있도록 합니다. 이는 CUDA의 광범위한 채택과 활용을 촉진

CUDA: New Features and Beyond(2024)

■ nvmath-python ← CUDA 수학 라이브러리 기능에 대한 Python의 손쉬운 접근성



CUDA: New Features and Beyond(2024)

■ CUTLASS Python Interface ←DL 프레임워크(예: PyTorch)로의 간편한 통합

C++ Interface

```
using Gemm =
typename gemm::kernel::DefaultGemmUniversal<
    half_t, layout::RowMajor,
    ComplexTransform::kNone, 8,
    half_t, layout::RowMajor,
    ComplexTransform::kNone, 8,
    half_t, layout::RowMajor,
    half_t, arch::OpClassTensorOp, arch::Sm80,
    gemm::GemmShape<256, 128, 64>,
    gemm::GemmShape<64, 64, 64>,
    gemm::GemmShape<16, 8, 16>,
    epilogue::thread::LinearCombination<
        half_t, 8, half_t, half_t>,
    gemm::threadblock::GemmIdentityThreadblockSwizzle<1>,
    3, arch::OpMultiplyAdd
>::GemmKernel;
```

Python Interface

```
// Create GEMM plan
plan = cutlass.op.Gemm(element=torch.float16,
                      layout=cutlass.LayoutType.RowMajor)

// Change swizzling functor
plan.swizzling_functor =
    cutlass.swizzle.ThreadblockSwizzleStreamK

// Add fused ReLU
plan.activation = cutlass.epilogue.relu
```

```
plan = cutlass.GroupedGemm(element=torch.float16,
                          layout=cutlass.LayoutType.RowMajor)

cutlass.emit.pytorch(plan.construct(), name='grouped_gemm',
                    cc=80, sourcedir='out')
```

out/setup.py

Python

out/grouped_gemm.cpp

C++

out/grouped_gemm.cu

CUDA

```
pip install nvidia-cutlass
```

```
# Import and run generated PyTorch extension
import grouped_gemm
Ds = grouped_gemm.run([A0, A1],
                     [B0, B1])
```

CUDA: New Features and Beyond(2024)

■ Python Developer Tools ← Python 기반 GPU 및 CPU 실행에 대한 자세한 검사

Python Call Stacks Samples and CUDA API Backtrace

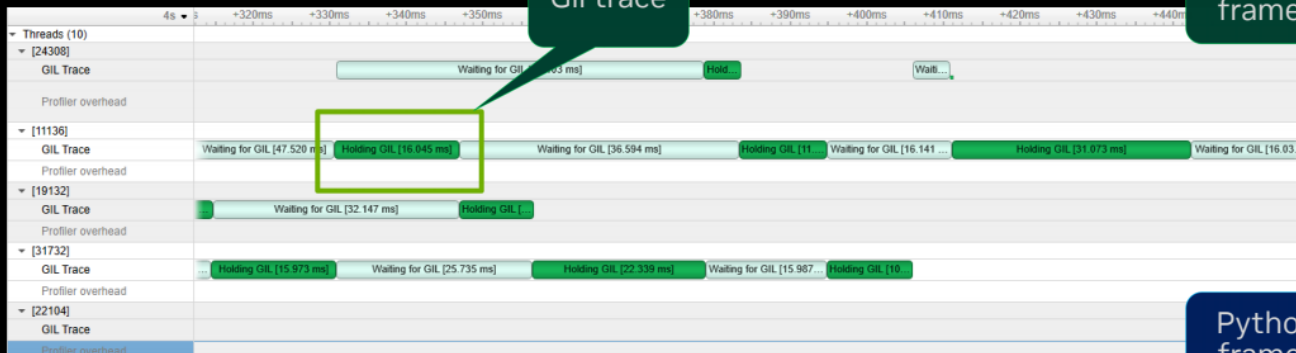
Identify where you are and how you got there

Global Interpreter Lock (GIL) trace

Common performance limiter in Python

Annotated dynamic code ranges

Built into in popular frameworks and libraries such as RAPIDS, Spark, CV-CUDA, and more...



Timeline range for a CUDA API call

C / C++ frames

Python frames

